DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

Academic Year 2026-2027 -ODD SEMESTER

Skill-Based Evaluation Report

(III Internal Assessment)

for

23AI2011

SOFTWARE ENGINEERING FOR ARTIFICIAL INTELLIGENCE

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE/ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Submitted by

K V BHUVANESH (URK23CS5075)

October 2026

Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
MoE, UGC & AICTE Approved
NAAC A++ Accredited

Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
MoE, UGC & AICTE Approved
NAAC A++ Accredited

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify that the skill-based evaluation report for the course **23AI2011 Software Engineering for Artificial Intelligence** is a bonafide work done during the odd semester of the academic year 2026-2027 by

K V BHUVANESH (Reg.No: URK23CS5075)

in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence / Artificial Intelligence & Machine Learning) of Karunya Institute of Technology and Sciences.

Signature of Course In-charge
Mr. K. Martin Victor
Assistant Professor

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

S. No.	Evaluation Parameter	Max Marks	Marks Secured
1.	Software Requirement Specification	10	
2.	Design document	15	
3.	Certification Course	5	
4.	Viva Voce	10	
		Total	**____/40**

Signature of Course In-charge
Mr. K. Martin Victor
Assistant Professor

Table of Contents

1. Project Overview 5
2. Software Requirement Specification 7
3. Design Report 13
4. Platform details 28
5. Implementation steps (Mention your steps) 30
6. Certificate 33
7. Results and Outputs 34
8. Conclusions 43
9. References 45

1. Project Overview

1.1 About the Project

This project, titled **"Brain Tumor MRI Classification Using Support Vector Machine (SVM)"**, studies how well a classical machine-learning pipeline can recognise four classes of brain-MRI slices (glioma, meningioma, pituitary tumour and no tumour) from a public dataset, and delivers the completed work as a tested web application named **Brain MRI SVM**. Each image is converted to grayscale, resized to 128×128 and described by 8,100 Histogram-of-Oriented-Gradients (HOG) features; a standardiser and a Support Vector Machine with an RBF kernel then assign one of the four classes.

Before any model was trained, the dataset was audited for duplicates. The audit found that 1,076 of the 1,600 images in the original Testing folder had a near-identical (often mirrored) copy in the Training folder. The 7,200 images were therefore grouped into 3,479 near-duplicate clusters, 207 images were quarantined and a cluster-disjoint split of 3,295 unique images (2,311 train, 492 validation, 492 test) was built, so that the reported scores are not inflated by copies of test images. Six candidate models (three HOG variants × linear or RBF kernel) were compared with cross-validation on the training partition and on the validation partition; the selected model was then evaluated once on the untouched test partition.

Brain MRI SVM is a React + TypeScript frontend with a FastAPI + scikit-learn backend. It displays the saved experiment results and classifies an uploaded image with the saved SVM, with research-prototype warnings on every page.

1.2 Problem Statement

Brain-tumour MRI must be read by specialists, the workload is high, and many published classifiers are hard to trust: they are evaluated on test images that have near-identical copies in the training data, they are tuned on the test set, and their code and models stay locked inside scripts and notebooks. Two questions are rarely answered fairly: (i) how accurate is a simple, fast and explainable baseline such as HOG + SVM once near-duplicate images between training and test data are removed and the test set is used only once, and (ii) can such a model be put into a validated software application that non-programmers can explore? This project answers both questions.

1.3 Objectives

- **O1.** Audit the dataset for exact and near-duplicate images and build a cluster-disjoint train/validation/test split with zero clusters crossing partitions.
- **O2.** Extract HOG features and train linear and RBF Support Vector Machines on the training partition only, with the scaler fitted on training data only.
- **O3.** Select the HOG variant, kernel and hyper-parameters with 5-fold cross-validation on the training partition and the validation partition, without reading the test set.
- **O4.** Evaluate the frozen model once on the 492 test images: accuracy, macro precision/recall/F1, confusion matrix and 95% bootstrap intervals, against a majority-class

baseline.

- **O5.** Re-measure the leakage controls (clusters, byte-identical files and nearest-training similarity) and state what the results do and do not support.
- **O6.** Design, build and test Brain MRI SVM following a documented SRS and design (Sections 2 and 3).

1.4 Real-World Application and Purpose

- **Healthcare AI research:** a fair, leakage-controlled baseline for four-class brain-tumour MRI classification against which deep-learning models can be compared.
- **Medical-image data engineering:** a reproducible near-duplicate audit and cluster-disjoint split that any imaging team can reuse before trusting a public dataset.
- **AI software engineering:** an example of turning a trained model and its results into a validated API and a browser interface, with requirements, UML design, automated tests and traceability.
- **Education and demonstration:** evaluators and students can explore results and try the classifier without writing code.

The project relates to UN SDG 3 (Good Health and Well-being) and SDG 9 (Industry, Innovation and Infrastructure) as research direction only. **Brain MRI SVM is an academic research prototype, not a medical device**; it has not been clinically validated and must not be used for diagnosis or treatment.

2. Software Requirement Specification

This Software Requirements Specification describes Brain MRI SVM version 1.0 as it was built and tested. Its structure follows IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018, and every functional requirement is traced to the implementation and to a test (Table 2.4).

2.1 Product Scope

Brain MRI SVM is a local, two-tier web application that (i) presents the saved results of the SVM experiment on the de-duplicated split (model selection, test metrics, confusion matrix, leakage evidence and figures) and (ii) classifies an uploaded PNG or JPEG brain-MRI image with the saved HOG + SVM model. The application only reads the experiment outputs and the model file; it never writes to them, so the research results remain reproducible.

Out of scope: training or re-training models from the interface, user accounts and log-in, storing uploaded images or any patient data, clinical decision support and cloud hosting. The product is intended for researchers, students and course evaluators, and it states its research-prototype status on every page.

2.2 User Classes and Characteristics

There are three kinds of users for this product:

- **Researcher / student (primary):** understands MRI slices and basic machine learning; uses all four pages to explore results and the model; no programming is needed.
- **Evaluator / faculty reviewer:** reads the results and tries the classification demonstration during the presentation; needs the limitations stated clearly.
- **Developer / maintainer:** starts the servers with *run_dev.sh*, runs the test suites, sets environment variables and uses the health and status endpoints.

2.3 System Modules

The system consists of the following modules:

- **Dashboard module:** key saved results (test accuracy, macro-F1, selected model, test-set size), model-selection and per-class charts, and model readiness.
- **Classification module:** image upload, validation and classification with the HOG + SVM model, with decision-score bars and caveats.
- **SVM results module:** test-set metrics with bootstrap intervals, confusion matrix, per-class results, model-selection chart, method, dataset notes and limitations.
- **Figures module:** saved figure sheets from the saved outputs, each labelled diagram, result, data or audit.
- **Backend service module:** FastAPI routes, request validation, the model service, result readers and configuration (*main.py, ml.py, results.py, config.py*).

2.4 Functional Requirements

FR1: Upload an MRI image
Description: The user selects a brain-MRI slice by drag-and-drop or the file picker on the Classify page; a preview is shown before the image is submitted.
Input: PNG or JPEG image file
Output: Image preview on the page

FR2: Validate the uploaded file
Description: The browser and the server check the type (PNG/JPEG), that the file is not empty, a size of at most 10 MB, sides between 32 and 8,192 pixels and that the image can be decoded.
Input: Uploaded file
Output: File accepted, or a specific error code (400, 413, 415 or 422) with a message

FR3: Classify an MRI image
Description: The server converts the image to grayscale, resizes it to 128×128, extracts 8,100 HOG features, standardises them and runs the RBF SVM over four classes.
Input: Validated image
Output: Predicted class, four decision scores, score margin, inference time

FR4: Show classification caveats
Description: Every result is shown with caveats: decision scores are not calibrated probabilities, the measured performance belongs to the 492-image test partition, there is no out-of-distribution check, patient-level independence is unproven and the output is not a diagnosis.
Input: Classification response
Output: Result panel with score bars and caveats

FR5: Show decision scores
Description: The page shows the four one-vs-rest decision scores as bars with the predicted class highlighted, the margin over the runner-up, the model name, kernel, input size and time.
Input: Classification response
Output: Score bars and model line

FR6: View test-set metrics
Description: The results page shows accuracy, balanced accuracy, macro precision, recall and F1, weighted F1, 95% bootstrap intervals and the majority-class baseline, read unchanged from the saved files.
Input: GET /api/metrics
Output: Metrics table

FR7: View confusion matrix and per-class results
Description: The results page shows the 4×4 confusion matrix and per-class precision, recall, F1 and one-vs-rest specificity, with the tumour-versus-no-tumour sensitivity and specificity.
Input: GET /api/metrics
Output: Matrix and per-class table

FR8: View model-selection results
Description: Cross-validation and validation macro-F1 of the six candidate models (three HOG variants × two kernels) are shown as a chart with the selection rule.
Input: GET /api/metrics
Output: Candidate chart

FR9: View dashboard
Description: The dashboard shows test accuracy, macro-F1, selected model, test-set size, the model-selection and per-class charts, model readiness and the limits to keep in mind.
Input: GET /api/metrics, /api/models/status, /api/experiment
Output: Key-figure tiles, charts and readiness card

FR10: View method, data and limitations
Description: The results page shows the method steps, model configuration, dataset and split description, leakage evidence and limitations read from the saved files.
Input: GET /api/experiment
Output: Notes and limitation list

FR11: Browse the figures
Description: Saved figure sheets are shown, each labelled diagram, result, data or audit, and served only from a fixed registry of known files.
Input: Figure identifier
Output: Labelled figure sheet

FR12: Report health and model status
Description: The system reports liveness, whether the classifier loaded, its kernel, support-vector count, SHA-256 and the reason for any unavailability.
Input: GET /api/health, GET /api/models/status
Output: JSON status with disclaimer

FR13: Load and verify the model
Description: The model is loaded once on first use behind a lock and verified (its SHA-256 must equal the hash recorded at the test evaluation; the class order is checked); a failing model is reported unavailable, never substituted.
Input: Model file and test evaluation
Output: Loaded model, or "unavailable" with the reason

FR14: Display research-prototype notice
Description: A notice is shown on every page, and a disclaimer is included in the classification and health responses.
Input: Any page or response
Output: Visible notice and disclaimer

FR15: Handle errors gracefully
Description: Every failure is returned as a structured JSON error and its message is shown without stopping the application, including when the backend cannot be reached.
Input: Any failed request
Output: JSON {error: {code, message}} shown in an error box

2.5 Non-Functional Requirements

NFR1: Performance: The system should answer a classification request within 5 seconds on the reference hardware once the model is loaded (observed: about 12 ms per image, 12.1 ms averaged over the 492 test images and 12.8 ms in the application).

NFR2: Security: The system should accept only PNG/JPEG uploads up to 10 MB, load the model only when its SHA-256 equals the evaluated model, allow cross-origin calls only from the local Vite origin, serve figures only from a fixed registry and contain no hard-coded user paths.

NFR3: Privacy and data protection: The system should process an uploaded image in memory only; it is not written to disk, not sent to any external service and its content is not logged.

NFR4: Reliability: The system should keep running when the model or a result file is missing or fails to load, report the affected feature as unavailable with the reason, and never substitute another model or invent a result.

NFR5: Accuracy of presented results: The system should show metric values exactly equal to the saved result files; nothing is recomputed or modified.

NFR6: Usability and accessibility: The system should be usable with keyboard and screen reader: labelled navigation, an upload area operable with Enter or Space, labelled controls and a visible busy state.

NFR7: Safety and ethics: The system should never present an output as a diagnosis; research status, score meaning and model limits should be visible wherever a result is shown.

NFR8: Portability: The system should run on macOS on the CPU (no GPU is needed), with all paths relative to the project root and configurable through environment variables.

NFR9: Maintainability and testability: The system should keep routes, the model service, result readers and configuration in separate modules, use a typed API client and keep an automated test suite.

2.6 Operating Environment and Constraints

Table 2.1: Operating environment.

Item	Environment
Client	A current desktop web browser; interface served at http://localhost:5173
Server	Python 3.14.3, FastAPI 0.143, Uvicorn 0.54, Pydantic 2.14, scikit-learn 1.9.1, scikit-image 0.26.0, NumPy 2.5.4, Pillow 12.2; listens on http://127.0.0.1:8000
Frontend build	React 19, TypeScript 6.0, Vite 8.3, React Router 7.18, Tailwind CSS 4.3 with shadcn/ui, Recharts 3.10
Hardware	Apple MacBook Air (M4, 16 GB unified memory), macOS; CPU only, no GPU is required

- The application runs locally; both servers bind to the local machine.

- The classifier works on 128×128 grayscale input derived from any uploaded image; it returns one of four labels and four decision scores.
- Uploads are limited to PNG/JPEG, ≤ 10 MB, sides 32–8,192 px.
- The CC BY 4.0 dataset is not redistributed; the scikit-learn and scikit-image libraries are used under their BSD licences.
- Assumptions: the model file (with its recorded SHA-256), the model configuration and the saved result files are present and unchanged.

2.7 External Interface Requirements

Table 2.2: External interface requirements.

Interface	Requirement
User interface	Single-page web interface with four pages (Dashboard, Classify MRI, SVM results, Figures), a common header and a research-prototype notice on every page.
Hardware interface	No special hardware; the model runs on the CPU.
Software interface	A joblib model bundle (scikit-learn pipeline) loaded after an SHA-256 check; libraries as in Table 2.1.
Communication interface	JSON over HTTP between the browser (Vite proxy, port 5173) and the backend (port 8000); uploads as multipart/form-data; errors as {error: {code, message}}.

Table 2.3: API endpoints of the backend.

Endpoint	Responsibility
GET /api/health	Liveness and the research-prototype disclaimer
GET /api/models/status	Classifier availability, kernel, support vectors, model SHA-256
POST /api/classify	PNG/JPEG upload → predicted class, four decision scores, margin, timing, caveats
GET /api/metrics	Test metrics, confusion matrix, model-selection candidates read from the saved files
GET /api/experiment	Method, model configuration, dataset description, leakage evidence, limitations
GET /api/gallery (+ /image/{id})	Fixed registry of saved figure sheets (diagram / result / data / audit)

2.8 Requirements Traceability

Table 2.4: Traceability of functional requirements to implementation and tests.

FR	Implemented in	Verification
FR1	Classify page (drop zone, picker, preview)	Manual browser check
FR2	Classify page checks; POST /api/classify	test_classify_error_handling; smoke test
FR3	POST /api/classify; svm_lib.predict_image	test_classify_matches_the_library_prediction; test_classify_png_and_jpeg_uploads
FR4	Classify page; caveats field	Manual browser check (Fig. 7.2)
FR5	Classify page score bars	Manual browser check (Fig. 7.2); smoke test (four scores)
FR6	SVM results page; /api/metrics	test_metrics_equal_saved_files; smoke test
FR7	SVM results page; /api/metrics	test_metrics_equal_saved_files
FR8	Dashboard and results charts; /api/metrics	Manual browser check (Figs. 7.1, 7.3)
FR9	Dashboard; /api/metrics, /api/models/status	test_metrics_equal_saved_files; Manual (Fig. 7.1)
FR10	Results page; /api/experiment	test_experiment_lists_limitations_and_leakage_evidence
FR11	Figures page; /api/gallery	test_gallery_registry_only
FR12	/api/health, /api/models/status	test_health; test_status_real_model_matches_evaluated_file
FR13	SVMService.ensure; _load	test_unavailable_or_mismatched_model_is_reported_not_substituted
FR14	Layout notice; response disclaimers	test_health; manual inspection
FR15	Exception handlers in main.py; ApiError in api.ts	test_classify_error_handling

3. Design Report

This section presents the design of the project: the overall workflow and architecture, five UML diagrams (use-case, class, two sequence diagrams and an activity diagram), the SVM model design, the main algorithms and the user-interface design. It realises the requirements of Section 2.

3.1 Design Principles

- **Truthful presentation:** a result is read from a saved file or produced by the real, verified model; otherwise the feature is reported unavailable.
- **Read-only application:** the backend only reads experiment outputs and the model file.
- **Validate at the boundary:** type, size and dimensions are checked before the model runs.
- **Research limits are part of the interface:** notices and caveats are produced by the backend and shown wherever a result appears.

3.2 Experimental Workflow

Figure 3.1 shows the end-to-end pipeline. The 7,200 Kaggle images are audited for near-duplicates (32×32 cosine similarity over eight orientations, threshold 0.98), 207 problem images are quarantined and the 3,295 unique images are split 70/15/15 so that no cluster crosses a partition. The training partition is used for 5-fold cross-validated grid search of six candidate models; the validation partition selects the winner; the test partition is read once with the frozen model. The saved outputs feed the web application.

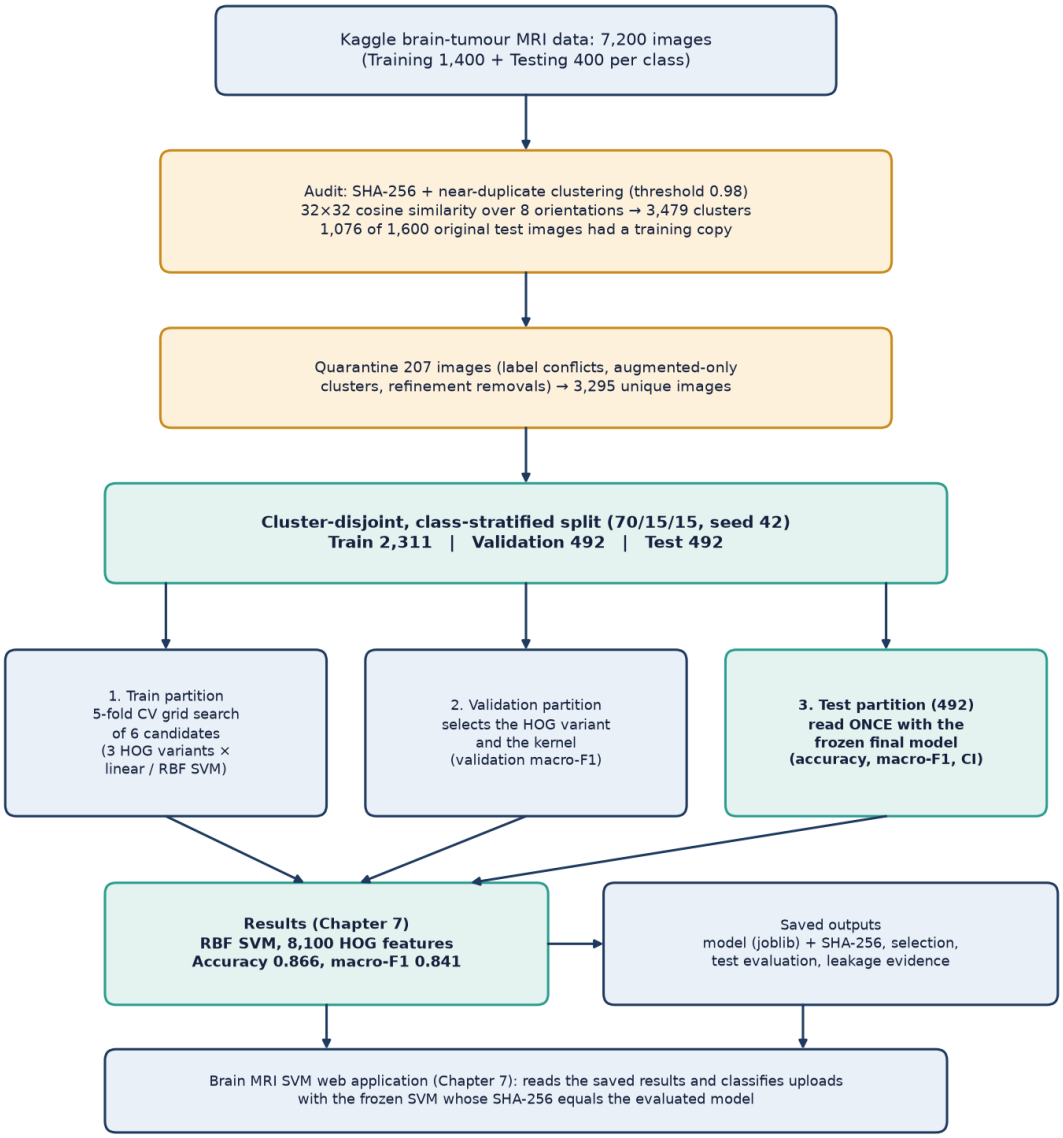


Figure 3.1: Experimental workflow of the project, from the original dataset to the results and the Brain MRI SVM application.

3.3 System Architecture

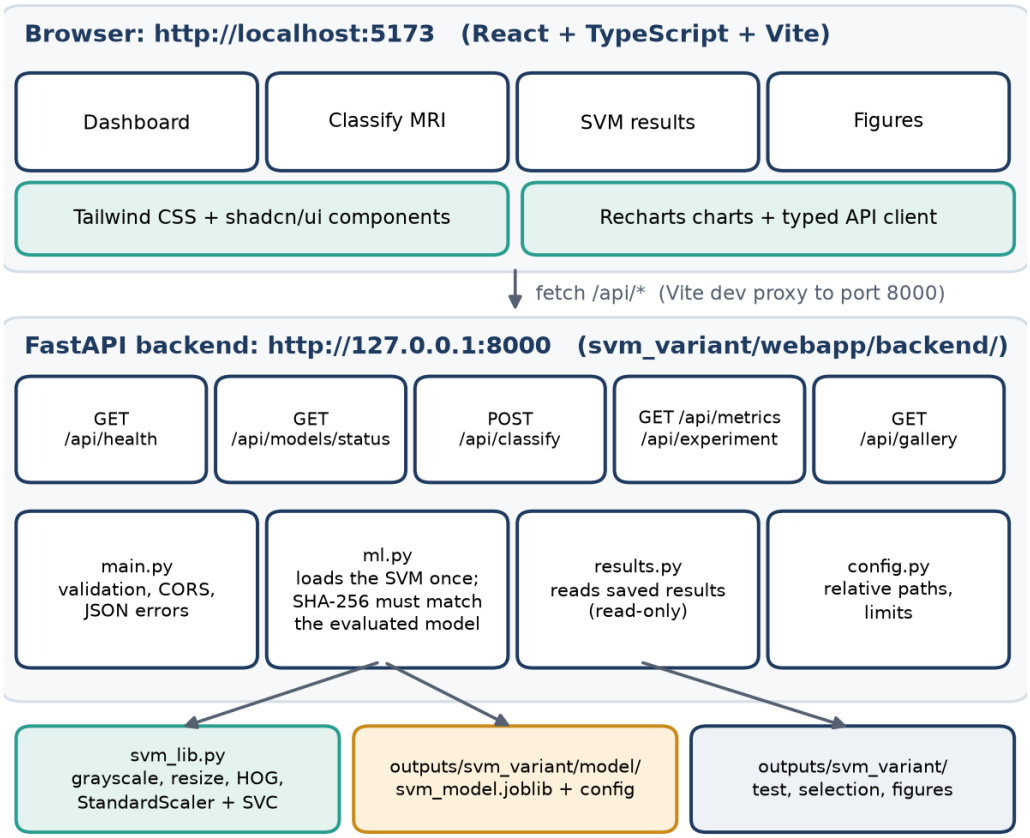


Figure 3.2: Software architecture of Brain MRI SVM (React frontend and FastAPI backend).

Figure 3.2 shows the two-tier architecture. The React + TypeScript single-page application (four pages, Tailwind/shadcn components, Recharts charts and a typed API client) calls the FastAPI backend through the Vite proxy. The backend has four modules: *main.py* (routes, validation, CORS, JSON errors), *ml.py* (the model service, loaded once and verified by SHA-256), *results.py* (read-only access to saved result files) and *config.py* (relative paths and limits). The saved SVM model bundle, its configuration and the saved experiment files are the only data sources.

Table 3.1: Main design decisions.

Decision	Reason
The model is loaded lazily, once, behind a lock	Fast start-up; safe under concurrent requests; a load error is stored and reported
Model file must have the SHA-256 recorded at the test evaluation	The application can only serve the exact model whose test results are shown
StandardScaler inside the pipeline, fitted on training data only; HOG is stateless	No information from validation or test data enters preprocessing
Test partition read once; a second evaluation is refused	The test score cannot be tuned against
Decision scores shown as scores, not probabilities	SVM scores are not calibrated; the page says so
Figures served from a fixed registry of identifiers	No user-supplied file path, so path traversal is impossible
One JSON error shape {error: {code, message}}	The client shows a precise message for every failure
Vite proxies /api; CORS allows only the Vite origin	The browser sees one origin; no other site can call the API

3.4 UML Diagrams

3.4.1 Use-Case Diagram

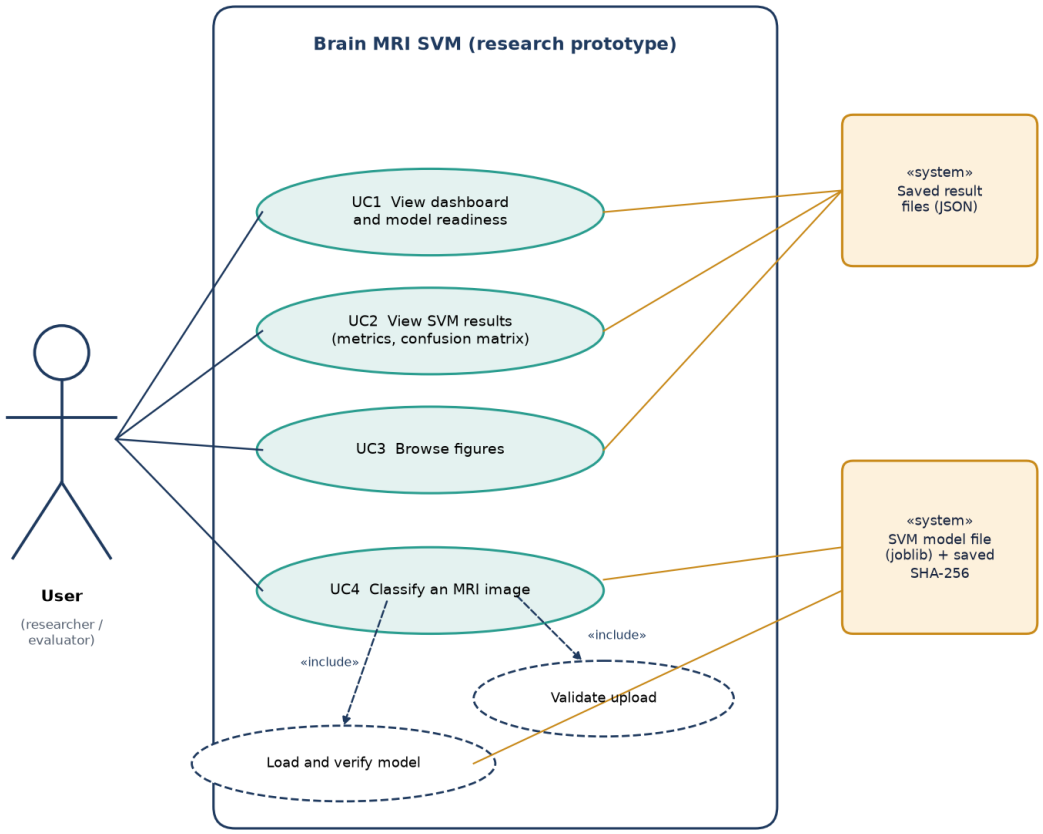


Figure 3.3: Use-case diagram of Brain MRI SVM.

Figure 3.3 shows the single human actor (researcher or evaluator) and four use cases: UC1 view dashboard and model readiness, UC2 view SVM results, UC3 browse figures and UC4

classify an MRI image. “Validate upload” and “Load and verify model” are «include» behaviours of UC4. Two system actors supply data: the saved result files and the SVM model file with its recorded SHA-256.

Table 3.2: Use-case specifications of the use cases that run the model and read results.

Field	UC4: Classify MRI image	UC2: View SVM results
Actors	User; SVM model file (system)	User; saved result files (system)
Precondition	Backend running; model file present, SHA-256 matches the evaluated model	Backend running; test_evaluation.json and selection.json present
Main flow	Choose image → browser checks → Classify → POST /api/classify → server validates, converts to 128×128 grayscale, HOG, scaler, SVM → class, four scores, margin, time, caveats shown	Open SVM results → GET /api/metrics and /api/experiment → tables, confusion matrix, charts and limitations shown
Alternative flows	Wrong type/size in browser; server error 400/413/415/422; model unavailable 503	Result file missing: 503 metrics_unavailable shown in an error box
Postcondition	Result displayed; image not stored	Values displayed exactly as saved

3.4.2 Class Diagram

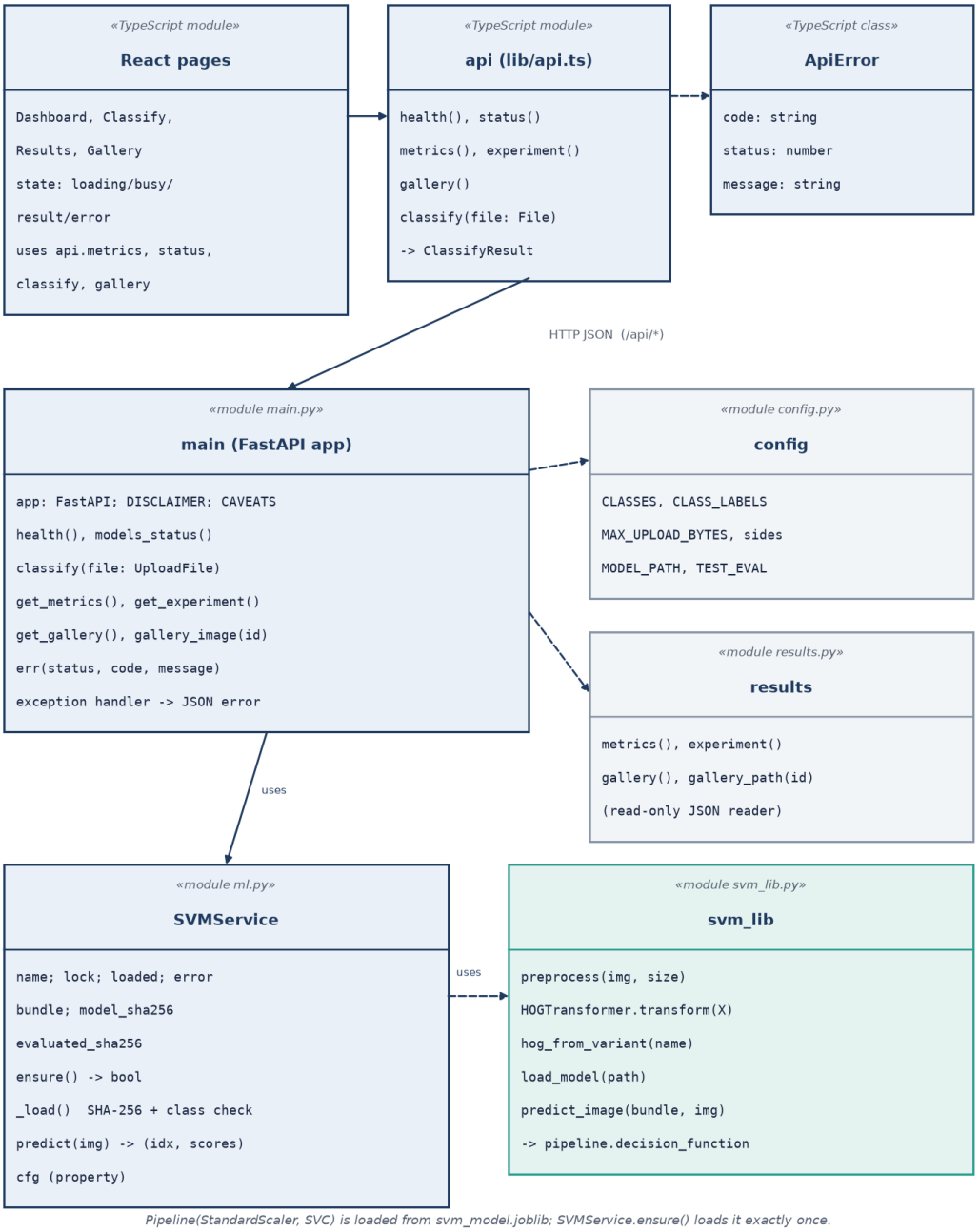


Figure 3.4: Class diagram of the frontend client, FastAPI module, result readers and the model service.

Figure 3.4 shows the static structure. The React pages use the typed client *api.ts*, which throws *ApiError* on failure. The FastAPI module *main* uses the results reader and the config constants. In *ml.py*, the class *SVMService* holds the lock, the loaded flag, the stored error, the model bundle and the two SHA-256 values, and its *ensure()* method loads and verifies the model exactly once. Prediction is delegated to *svm_lib*, which holds the preprocessing, the stateless *HOGTransformer* and the saved scikit-learn pipeline (StandardScaler + SVC).

3.4.3 Sequence Diagrams

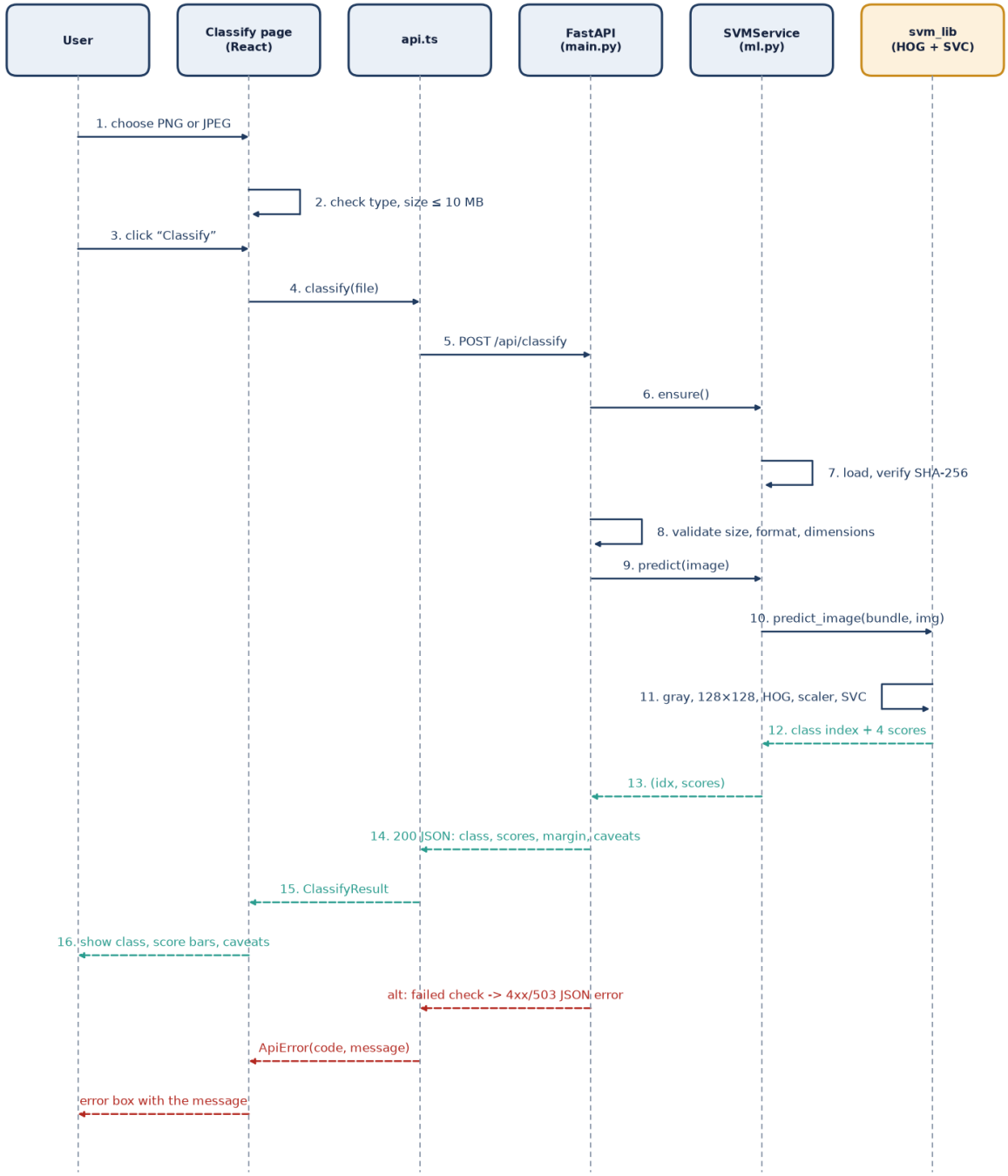


Figure 3.5: Sequence diagram of the classification request, including the one-time model load and the error path.

Figure 3.5 shows the message order for UC4. The page checks the file, the API client posts it, FastAPI calls *ensure()* (which loads and verifies the model only on the first request), validates the size, format and dimensions, and the model service pre-processes the image, extracts HOG features and calls the pipeline. The four decision scores are returned as JSON; any failed check returns a JSON error that the page shows in its error box.

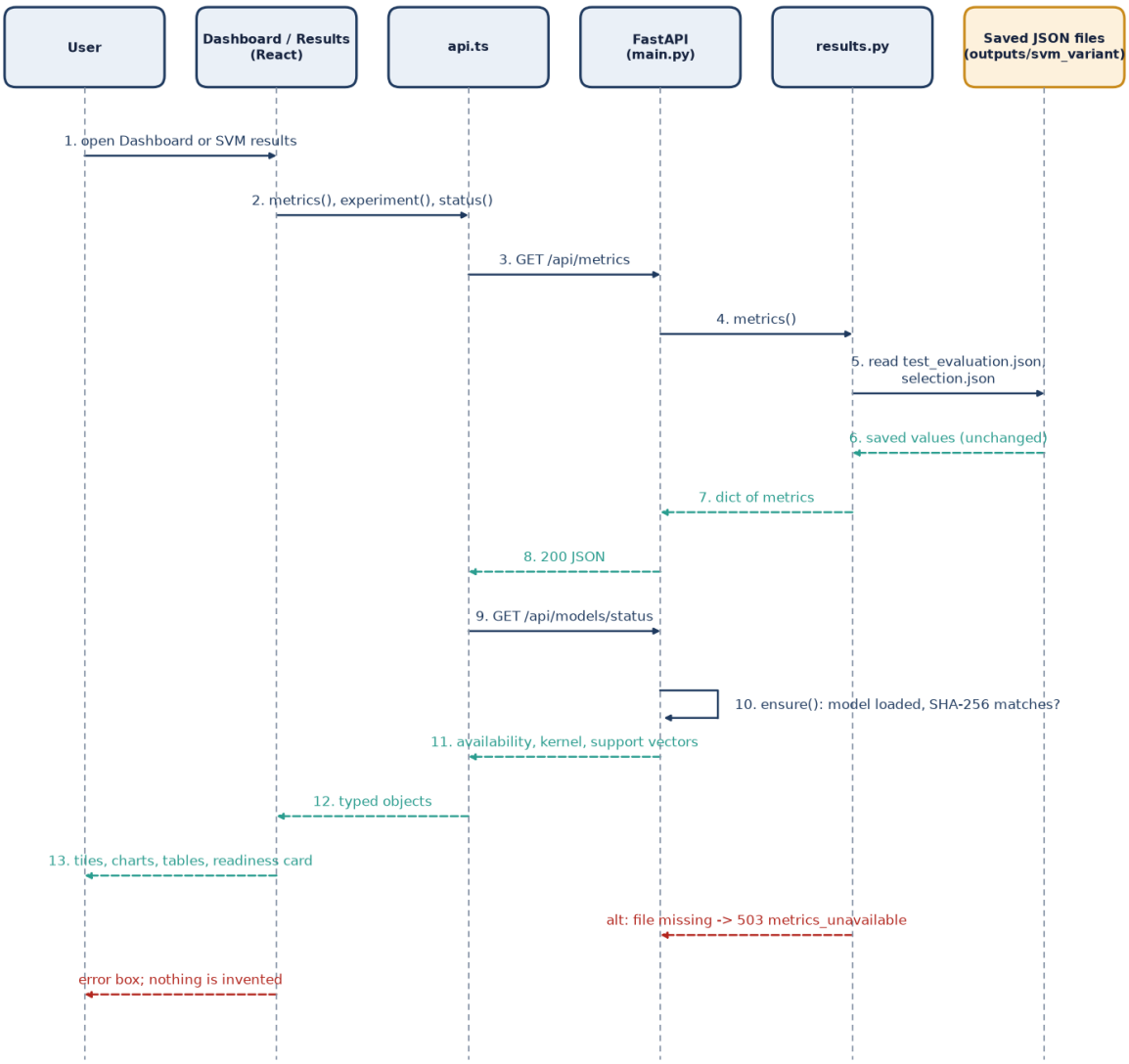


Figure 3.6: Sequence diagram of viewing the saved results on the Dashboard and SVM results pages.

Figure 3.6 shows UC1 and UC2. The page asks the typed client for metrics, experiment notes and model status; the backend reads the saved JSON files through *results.py*, passes the values through unchanged and checks that the model is available. If a saved file is missing, a 503 error is returned and shown; nothing is invented.

3.4.4 Activity Diagram

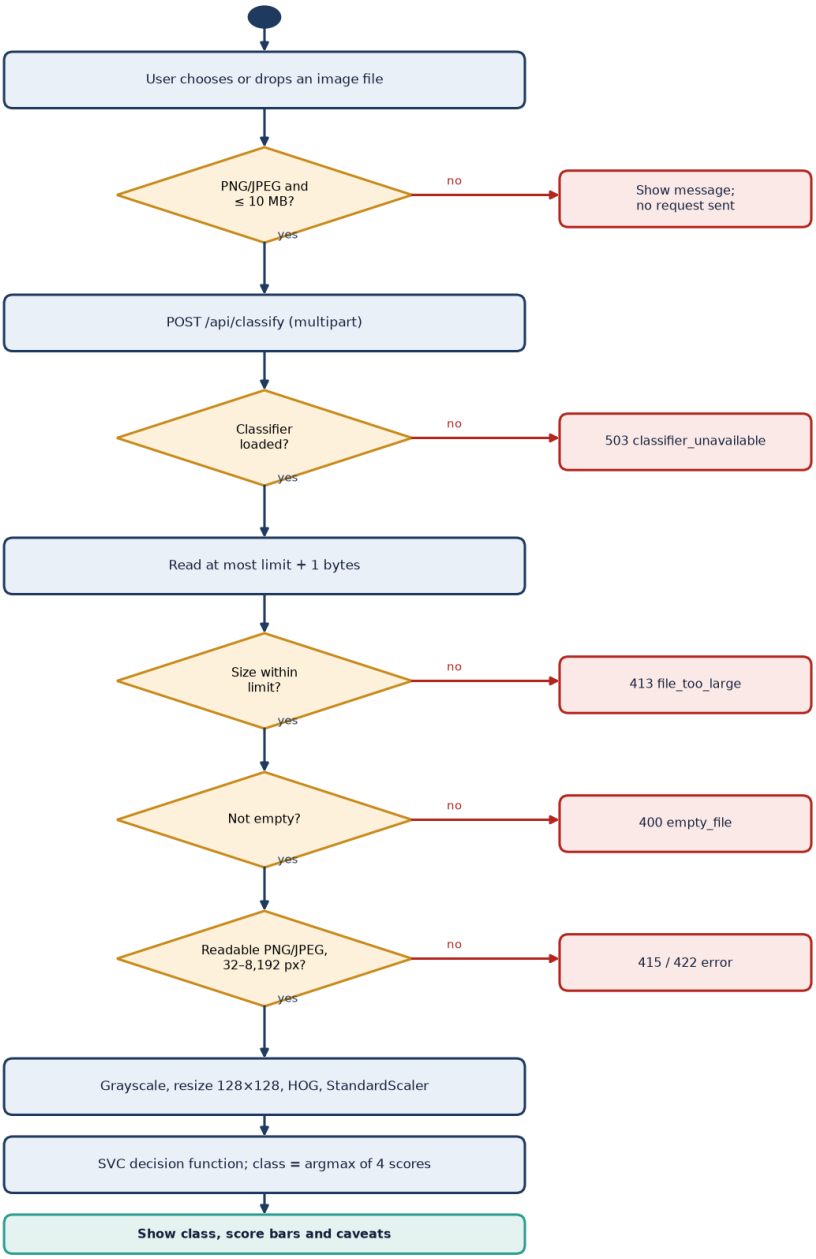


Figure 3.7: Activity diagram of an image upload: the checks that decide between an error message and a classification result.

Figure 3.7 shows the decision order for an upload. Model availability is checked first, so a request is refused before the file is read; the size check reads at most one byte more than the limit, so an oversized file is never held in memory in full. The file is then checked for emptiness, image format and dimensions before pre-processing, HOG extraction, the SVM and the response with caveats.

3.5 SVM Model Design

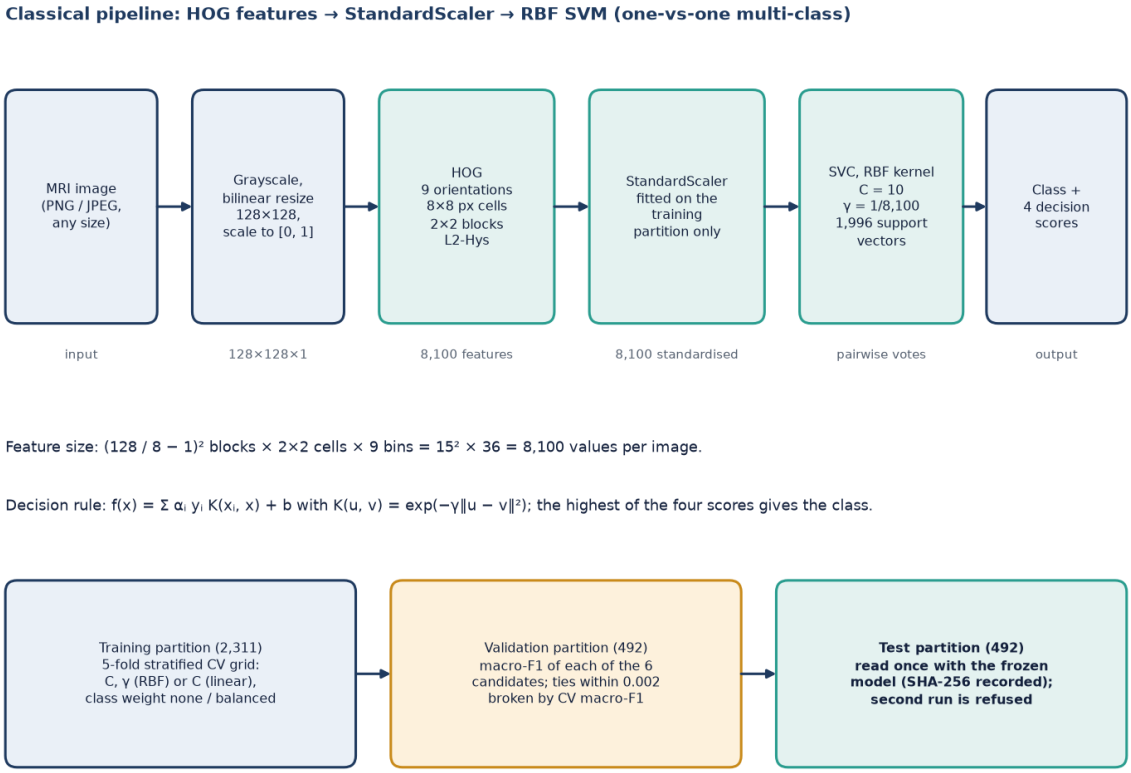


Figure 3.8: Design of the HOG + StandardScaler + RBF-SVM classifier and how the three partitions are used.

Figure 3.8 shows the proposed model. An image is converted to grayscale, resized bilinearly to 128×128 and scaled to [0, 1]. HOG divides it into 8×8-pixel cells, builds a 9-bin histogram of gradient orientations per cell and normalises 2×2-cell blocks with L2-Hys, giving $(16 - 1)^2 \times 36 = 8{,}100$ features. A StandardScaler fitted on the training partition only standardises each feature, and a Support Vector Machine with an RBF kernel $K(u, v) = \exp(-\gamma\|u - v\|^2)$, $C = 10$ and $\gamma = 1/8{,}100$ separates the four classes; the final model has 1,996 support vectors.

Table 3.3: Comparison of the two kernels that were tuned.

Item	Linear SVM	RBF SVM (selected)
Decision function	$w \cdot x + b$	$\Sigma\, \alpha_i\, y_i\, K(x_i, x) + b$ with the RBF kernel K
Hyper-parameters tuned	C in {1e-5 … 1}; class weight none / balanced	C in {1, 10, 100}; γ in {0.1, 1, 3}/n_features; class weight none / balanced
Grid size × 5 folds	12 settings × 5 = 60 fits per HOG variant	18 settings × 5 = 90 fits per HOG variant
Best setting (hog128_cell8)	C = 0.001, balanced	C = 10, γ = 1/8,100, no class weight
CV macro-F1 / validation macro-F1	0.834 / 0.866	0.846 / 0.886
Remark	Fast and simple; limited to a linear boundary	Handles non-linear class boundaries; more support vectors

3.6 Algorithms

Five algorithms are central to the project. Algorithms 3.1 and 3.2 belong to the research pipeline, 3.3 and 3.4 are executed by the application and 3.5 defines the evaluation. Symbols: τ is the similarity threshold, C the SVM penalty, γ the RBF width and n_f the number of HOG features.

Algorithm 3.1: Near-duplicate clustering and cluster-disjoint split

Input: pooled images P (7,200), threshold $\tau = 0.98$, ratios 70/15/15, seed 42
Output: partitions Train, Validation, Test with no cluster in more than one

1 for each image i: $v_i \leftarrow$ 32×32 grayscale, mean removed, length-normalised; $h_i \leftarrow$ SHA-256 of the file
2 for each pair (i, j): $s_{ij} \leftarrow$ max over 8 orientations o of $v_i \cdot o(v_j)$
3 if $s_{ij} \geq \tau$ or $h_i = h_j$: union(i, j)
4 clusters ← connected components of the union-find structure
5 quarantine clusters with conflicting class labels and clusters made only of "aug" files
6 rep(c) ← one deterministic representative per cluster (non-augmented, originally Training preferred)
7 for each class: assign representatives to Train / Validation / Test in the ratio 70/15/15
8 verify that no cluster occurs in more than one partition; write the manifest with file hashes

Cost $O(8 \cdot n^2 \cdot d)$ with $d = 1{,}024$, computed as matrix products. It finds copies, mirrors and re-encodings, not different slices of the same patient. The SVM uses the representatives only.

Algorithm 3.2: HOG + SVM training and model selection

Input: Train (2,311) and Validation (492) images, three HOG variants, kernels {linear, RBF}, seed 42
Output: frozen pipeline (StandardScaler → SVC) with its configuration

1 for each variant v in {hog128_cell16, hog128_cell8, hog64_cell8}: X ← HOG(grayscale, resize, [0, 1]) for Train and Validation
2 for each kernel k in {linear, RBF}: grid search with 5-fold stratified CV on Train only
3 the pipeline StandardScaler → SVC is refitted inside every fold (scaler sees training folds only)
4 linear: C in {1e-5 … 1}; RBF: C in {1, 10, 100}, γ in $\{0.1, 1, 3\}/n_f$; class weight in {none, balanced}
5 best ← highest CV macro-F1; refit on Train; record CV and validation macro-F1
6 select the candidate with the highest validation macro-F1 (ties within 0.002: higher CV macro-F1)
7 refit the selected pipeline on Train only; save the model, its configuration and the file-list hashes
8 the test partition is not read in this algorithm

450 fits for six candidates, 781 s of grid search and 7.7 s for the final fit on the CPU. The first search was stopped because the linear optimum lay at the lower edge of its C grid; the grid was extended (C from 1e-5) and the search re-run from scratch.

Algorithm 3.3: Validation and classification of an uploaded image (application)

Input: uploaded file
Output: predicted class with four decision scores, or a JSON error

1 if the model is not loaded and cannot be loaded: error 503
2 data ← read at most (limit + 1) bytes; if longer than the limit: error 413; if empty: error 400
3 open data as an image; if unreadable: error 415 (not an image); if not PNG/JPEG: error 415
4 if the shorter side < 32 or the longer side > 8,192 pixels: error 422
5 g ← grayscale, resized to 128×128 (bilinear), scaled to [0, 1]
6 h ← HOG(g) (8,100 features); z ← StandardScaler(h)
7 s ← SVC decision function (four scores); class ← argmax s
8 return class, all four s, margin = s(1) − s(2), time, disclaimer and caveats

One request takes about 12 ms. There is no out-of-distribution test; any readable image receives one of four labels.

Algorithm 3.4: Loading and verifying the model (application)

Input: model file, test_evaluation.json
Output: loaded model bundle, or "unavailable" with the reason

1 acquire the lock; if the model is already loaded or an error is stored: return the stored state
2 if the model file is missing: error "run the training scripts first"
3 if test_evaluation.json is missing: error "the model has not been evaluated yet"
4 h ← SHA-256 of the model file; if h ≠ the hash recorded at the test evaluation: error (model differs from the evaluated model)
5 bundle ← load the pipeline and configuration (local, trusted file); if the class order differs from the application's: error
6 mark loaded; any exception is stored verbatim and reported, never replaced by another model

The check guarantees that the model behind the Classify page is exactly the model whose test results are shown.

Algorithm 3.5: Single test evaluation and metrics

Input: frozen model, test partition (492 images), saved model hash
Output: metrics, confusion matrix and bootstrap intervals

1 refuse to run if the test evaluation already exists; verify that the test files are disjoint from Train and Validation
2 predict all 492 images; build the confusion matrix C (rows = true, columns = predicted)
3 accuracy, per-class precision / recall / F1, macro averages, balanced accuracy and weighted F1 from C
4 one-vs-rest sensitivity and specificity per class; tumour-versus-no-tumour sensitivity and specificity
5 bootstrap: 2,000 resamples of the test predictions (seed 42); 95% percentile intervals for accuracy and macro-F1
6 majority-class baseline (always predict glioma) for reference
7 write the results with the SHA-256 of the model file

The test partition is evaluated once; the saved hash lets the application prove that it serves the evaluated model.

3.7 User Interface Design

The interface is a calm, data-first dashboard. Navy is used for structure, teal for emphasis and result panels, and amber for every research-prototype notice, so that a warning always has the same colour. Components come from shadcn/ui and charts from Recharts.

Table 3.4: Design tokens of the interface.

Token	Value	Use
Primary (navy)	#1F3A5F	Header, primary buttons, headings, chart bars
Accent (teal)	#2A9D8F	Selected states, result panels, score bars
Background / card	#F6F8FA / #FFFFFF	Page background / cards
Warning (amber)	#FDF1DC, border #C98A1A	Research-prototype notice and caveats
Destructive (red)	#B3261E	Error messages
Shape and type	radius 0.625 rem; Inter	Cards, buttons, all text

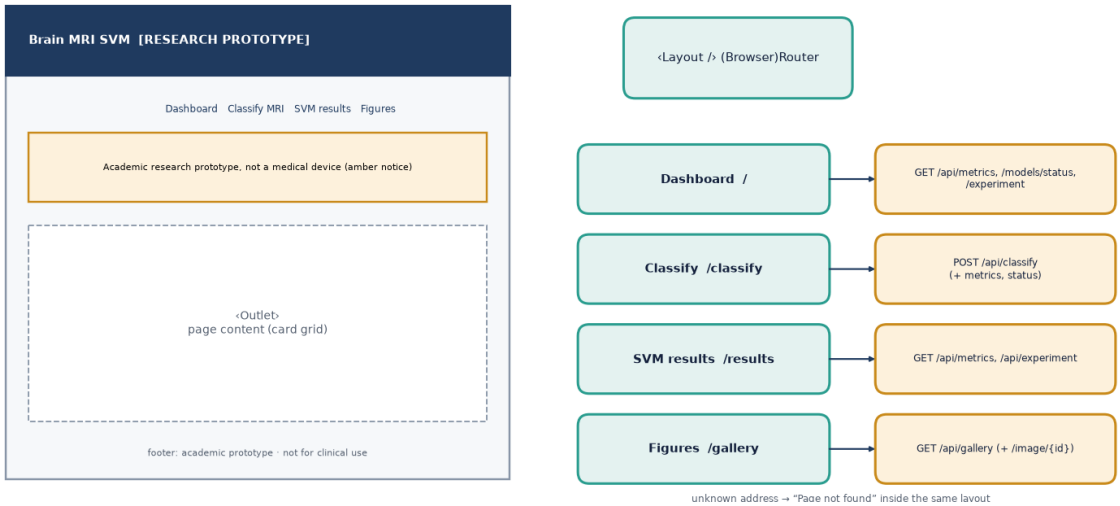


Figure 3.9: Common page layout (left) and route map with the API calls of each page (right).

Figure 3.9 shows the shared layout: a sticky navy header with the product name, a “research prototype” tag and navigation, the amber notice, the page content and a footer. The route map links each of the four routes to the API calls behind it; an unknown address shows “Page not found” inside the same layout.

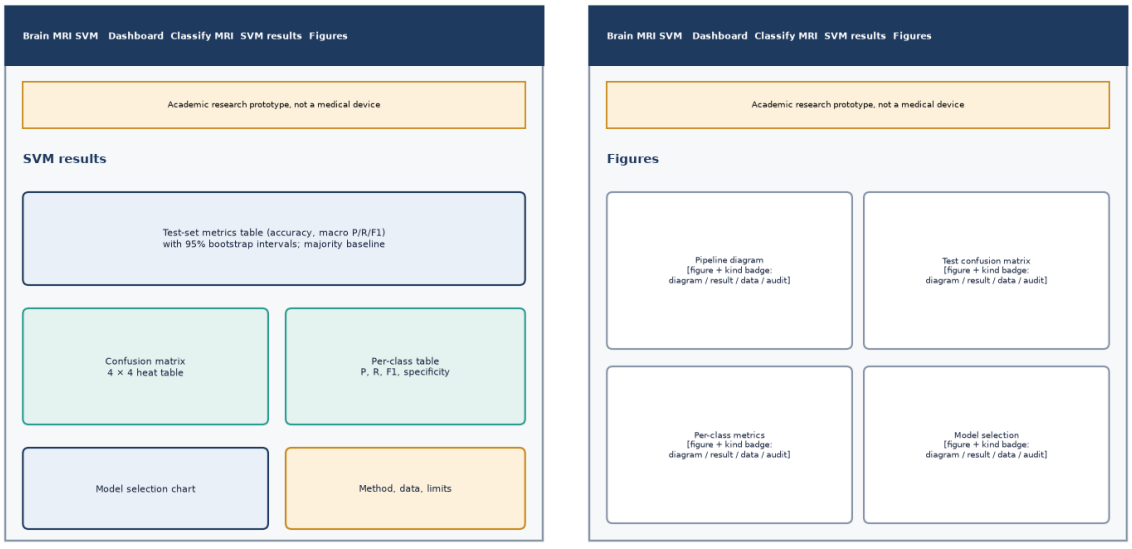


(a) Dashboard /

(b) Classify MRI /classify

Figure 3.10: Wireframes of the Dashboard and Classify MRI pages.

Figure 3.10 gives the first two wireframes. The Dashboard has four key-figure tiles, the model-selection and per-class charts and a model-readiness card. Classify MRI places the drop zone on the left and the model output with score bars and caveats on the right.



(c) SVM results /results

(d) Figures /gallery

Figure 3.11: Wireframes of the SVM results and Figures pages.

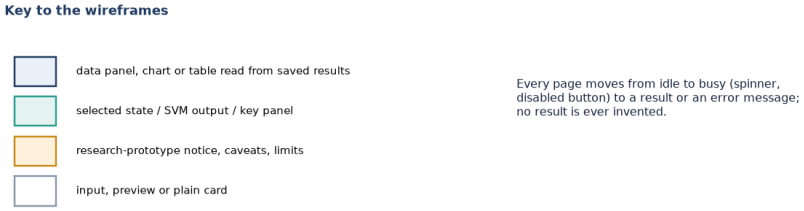


Figure 3.12: Key to the wireframes.

Figure 3.11 shows the SVM results page (metrics table, confusion matrix, per-class table, selection chart, method and limits), the Figures page and the colour key. Every page moves from idle to busy (spinner, disabled button) to either a result or an error message; no result is ever invented.

Table 3.5: Page designs: components, data source and states.

Page	Main components	Data source	States handled
Dashboard	Key-figure tiles; model-selection chart; per-class chart; readiness card; limits	GET /api/metrics, /api/models/status, /api/experiment	loading, error, model unavailable
Classify MRI	Drop zone with preview; Classify/Clear; result panel with score bars; caveats	POST /api/classify	idle, rejected, busy, result, error, unavailable
SVM results	Metrics table; confusion matrix; per-class table; charts; method and limitations	GET /api/metrics, /api/experiment	loading, error
Figures	Figure cards with kind badge and provenance note	GET /api/gallery	loading, error, empty

Accessibility and responsiveness: navigation and images carry labels, the drop zone is a focusable button opened with Enter or Space, the busy state is visible and the result area is announced politely. The navigation wraps, the two-column pages stack on narrow screens and the grids change their number of columns with the width.

4. Platform details

The project was developed and run entirely on a local machine; no cloud service is used at run time, which keeps any uploaded image on the user's computer.

Table 4.1: Tools and platform used.

Layer	Tools used (version)
Hardware / OS	Apple MacBook Air, M4 chip, 16 GB unified memory, macOS (CPU only)
Language	Python 3.14.3 (research and backend); TypeScript 6.0 (frontend)
Machine learning	scikit-learn 1.9.1 (StandardScaler, SVC, GridSearchCV, metrics), scikit-image 0.26.0 (HOG), NumPy 2.5.4, SciPy 1.18, joblib 1.6, pandas
Backend API	FastAPI 0.143, Uvicorn 0.54, Pydantic 2.14, Pillow 12.2
Frontend	React 19, Vite 8.3, React Router 7.18, Tailwind CSS 4.3, shadcn/ui (Radix, lucide), Recharts 3.10
Testing	pytest 9.1 with the FastAPI test client; shell integration smoke test; 62-check QA script; TypeScript/Vite production build
Dataset	Brain Tumor MRI Dataset (M. Nickparvar, Kaggle, Version 2, CC BY 4.0) — 7,200 images, four classes
Data split	Cluster-disjoint manifest: 2,311 train / 492 validation / 492 test representative images (hash-recorded)

Justification

- **scikit-learn + scikit-image:** standard, well-tested implementations of the SVM, cross-validated grid search, macro-F1 and HOG, so the results are reproducible and comparable with the literature.
- **HOG + SVM on the CPU:** a small, explainable model that trains in seconds, classifies an image in about 12 ms and needs no GPU.
- **FastAPI + Pydantic:** fast Python API that shares the scikit-learn environment, validates requests automatically and generates interactive API documentation (/docs).
- **React + TypeScript + Vite:** typed, component-based single-page interface with a fast development server and proxy, and a type-checked production build.
- **Local deployment:** privacy (images never leave the machine), zero hosting cost, full reproducibility and scalability later by containerising the same backend.
- **Open-source stack:** all tools are free, widely used and well documented, which keeps the project cost-effective and maintainable.

Table 4.2: Training configuration and runtime.

Item	Setting
Training data	The 2,311-image training partition only (file-list hash recorded); validation (492) used for selection only
Pre-processing	Grayscale, bilinear resize to 128×128 (64×64 for one variant), scale to [0, 1]; HOG with 9 orientations, 8×8 or 16×16 cells, 2×2 blocks, L2-Hys
Search	5-fold stratified cross-validation on the training partition, seed 42; six candidates; 450 fits
Selected model	RBF SVM, hog128_cell8 (8,100 features), $C = 10$, $\gamma = 1/8{,}100$, no class weight; 1,996 support vectors
Runtime on the CPU	Grid search 781 s in total (405 s for the selected candidate); final fit 7.7 s; HOG of the 492 test images 0.77 s; 12.1 ms per image in total
Interrupted run	The first search was stopped because the linear optimum lay at the lower edge of its C grid; the grid was extended and the whole search re-run from scratch (train and validation data only)

5. Implementation steps

1. **Collected the dataset.** Downloaded the four-class Brain Tumor MRI Dataset (Training and Testing folders, 1,400 + 400 images per class, 7,200 files) from Kaggle and recorded its licence and provenance.
2. **Audited the data for duplicates.** Found 153 groups of byte-identical files with SHA-256 hashing, then detected mirrored, rotated and re-encoded copies with 32×32 cosine similarity over eight orientations (threshold 0.98), followed by a 96×96 refinement stage (Algorithm 3.1).
3. **Built the cluster-disjoint split.** Formed 3,479 clusters, quarantined 207 images (conflicting labels, augmented-only clusters, refinement removals) and split the 3,295 representatives 70/15/15 with seed 42 into 2,311 / 492 / 492 images; the manifest is hash-recorded.
4. **Pre-processed the images.** Converted to one-channel grayscale, resized bilinearly to 128×128 (64×64 for one variant) and scaled to [0, 1]; the same function is used for training, evaluation and the web application.
5. **Implemented the feature pipeline** in *svm_lib.py*: split mapping, preprocessing, a stateless HOG transformer with three variants and the single-image prediction helper.
6. **Selected the model** with *train_select.py*: 5-fold cross-validated grid search of linear and RBF SVMs for each HOG variant on the training partition, then selection on validation macro-F1 (Algorithm 3.2).
7. **Froze the model.** Refitted the winner on the training partition only and saved the model (joblib), its configuration and the file-list hashes.
8. **Evaluated once on the test partition** with *evaluate_test.py*: metrics, confusion matrix, 95% bootstrap intervals and the majority-class baseline; a second run is refused (Algorithm 3.5).
9. **Re-measured the leakage controls** with *leakage_evidence.py*: clusters across partitions, byte-identical files and nearest-training similarity.
10. **Verified the experiment** with 62 automated checks (*qa_check.py*): model reload, saved predictions, recomputed metrics, run order, leakage evidence and report numbers.
11. **Wrote the SRS and design** (Sections 2 and 3): requirements, UML diagrams, algorithms and wireframes.
12. **Built the FastAPI backend** (main.py, ml.py, results.py, config.py) with seven routes, lazy verified model loading, input validation and structured JSON errors.
13. **Built the React + TypeScript frontend** with four pages, the typed API client, charts and research-prototype notices.
14. **Tested the application:** 13 backend pytest tests, a 7-check integration smoke test, the production build and manual browser checks; both servers start with one command, *./svm_variant/webapp/run_dev.sh* (backend on port 8000, frontend on port 5173).

Table 5.1: Main research code components.

File	Responsibility
svm_lib.py	Split mapping, grayscale/resize/[0, 1] preprocessing, HOG transformer, model loading and single-image prediction
src/dedup_split.py, src/dedup_refine.py	Near-duplicate similarity, clustering, quarantine, cluster-disjoint split, refinement (shared data step)
train_select.py	Cross-validated grid search of six candidates, validation-based selection, final fit and saving
evaluate_test.py	One-time test evaluation, bootstrap intervals, saved predictions; refuses a second run
leakage_evidence.py	Cluster, byte-identity and nearest-training-similarity audit; run-order evidence
make_figures.py	Pipeline, HOG, class-distribution, selection, confusion-matrix and error figures
qa_check.py	62 automated integrity checks
webapp/backend/*.py	FastAPI routes, model service, result readers, configuration and tests

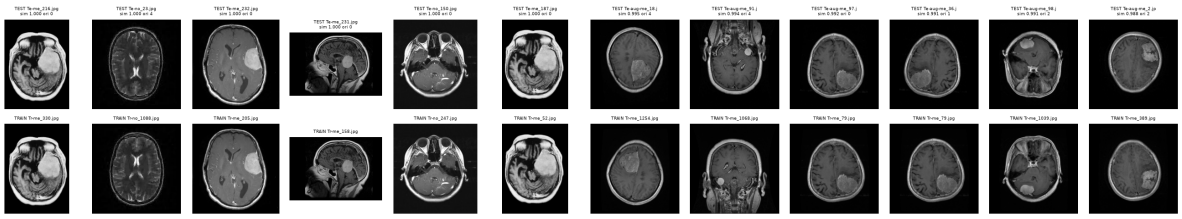


Figure 5.1: Examples from the audit: original Testing images (top) and their near-identical, often mirrored, copies in Training (bottom).

Figure 5.1 illustrates why step 2 was needed: the original folders contained test images whose mirrored copies were in training, which would have inflated every test score.

Table 5.2: Result of the duplicate audit and the final split.

Quantity	Value
Candidate images pooled	7,200
Original Testing images with a near-identical Training copy	1,076 of 1,600
Near-duplicate clusters (largest: 83 images)	3,479
Quarantined images	207
Final usable unique images	3,295
Train / validation / test	2,311 / 492 / 492
Clusters appearing in more than one partition	0

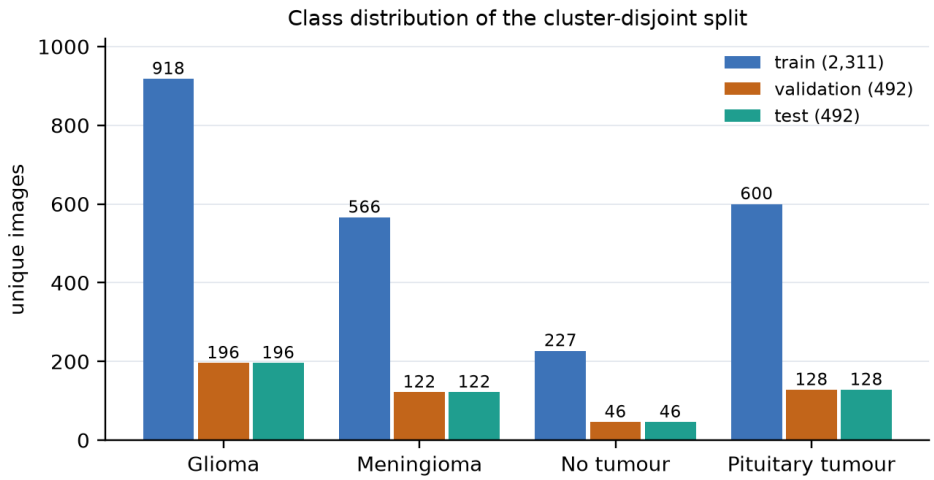


Figure 5.2: Unique images per partition and class after de-duplication.

Table 5.3: Application tests and outcomes.

Test	What it checks	Outcome
Backend pytest suite (13 tests)	Health and status; the real model loads and its SHA-256 equals the evaluated file; metrics equal saved files and are recomputable from saved predictions; classification equals the library prediction; PNG and JPEG uploads; invalid input and error cases; unavailable or mismatched model reported, not substituted; only the SVM classifier is exposed; figure path traversal blocked; CORS restricted; no hard-coded paths	13 passed
Integration smoke test (7 checks)	Page served, health, model available and matching, metrics equal the saved test file, classification of a real image (four scores), rejection of a text file, no other model endpoint	7/7 passed
QA script (62 checks)	Model reload, predictions and metrics recomputed, selection read only train/validation, model written before the test evaluation, second test evaluation refused, leakage evidence, report numbers	62/62 passed
Frontend production build	TypeScript type-check and Vite build	No errors
Manual browser checks	All four pages in a real browser	Worked as expected

6. Certificate

Certification course completed: **Software Development Lifecycle**, issued by **iCompetent Solutions** to **K V Bhuvanesh (URK23CS5075)**. Date of completion: 23 October 2026; assessment average: 89%; learning time: 5 h 56 min; certificate ID: CERT-2026-001565 (verifiable through the QR code on the certificate).

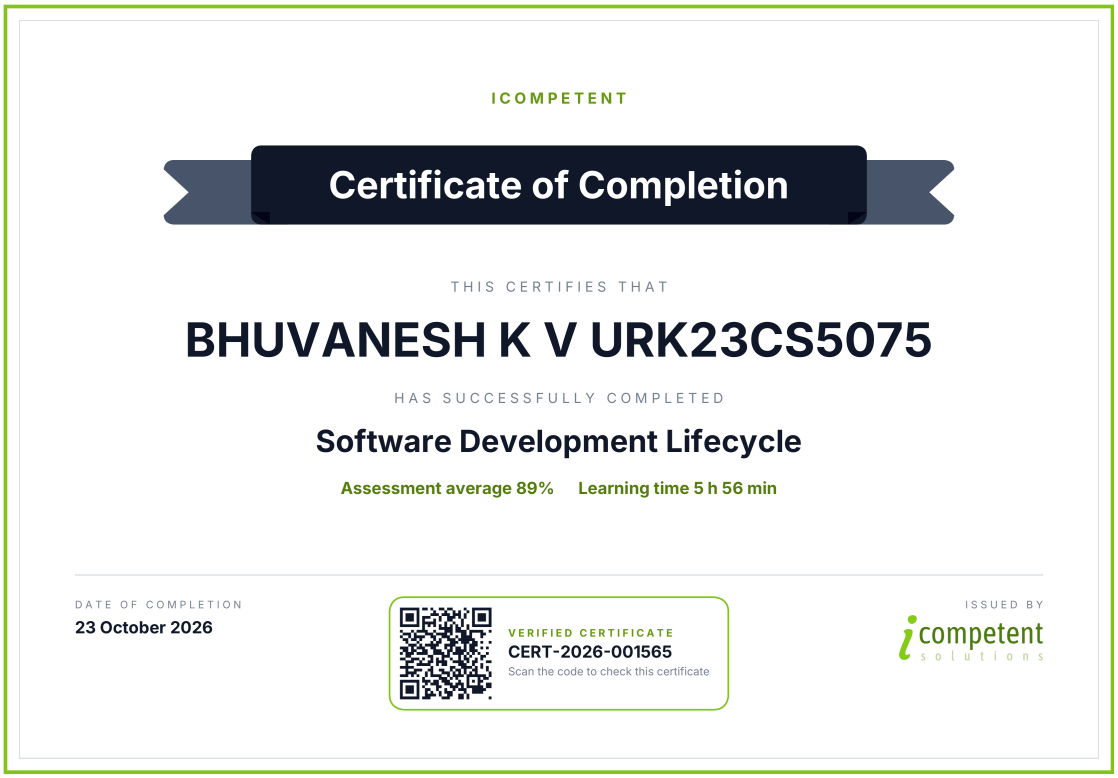


Figure 6.1: Certificate of completion — Software Development Lifecycle (iCompetent, CERT-2026-001565).

7. Results and Outputs

All numbers in this section come from the saved result files of the final SVM experiment on the de-duplicated split; the screenshots were captured from the running Brain MRI SVM application.

7.1 Web Application Outputs

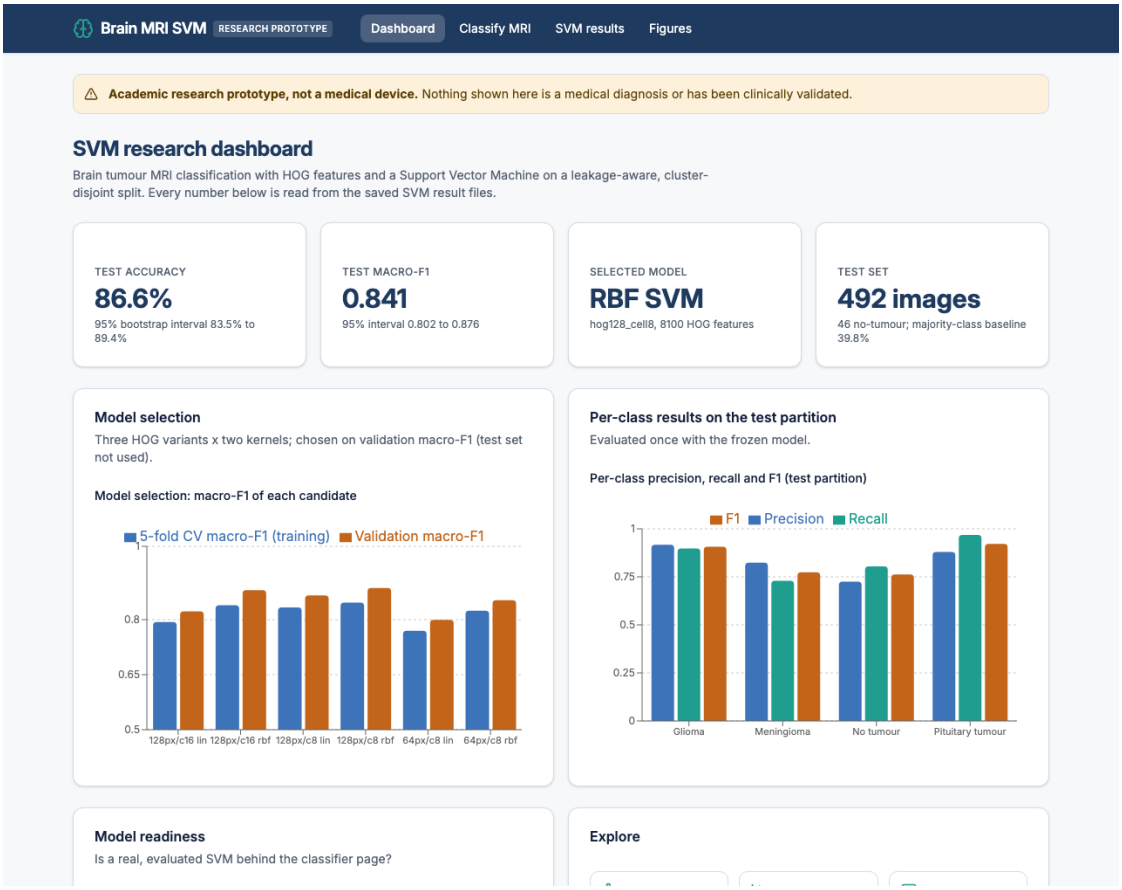


Figure 7.1: Brain MRI SVM research dashboard: test accuracy, macro-F1, selected model, charts and key figures.

The dashboard summarises the study at a glance: the selected RBF SVM reaches 86.6% test accuracy (95% bootstrap interval 83.5% to 89.4%) and macro-F1 0.841 on the 492-image test set, which contains only 46 no-tumour images; the majority-class baseline is 39.8%.

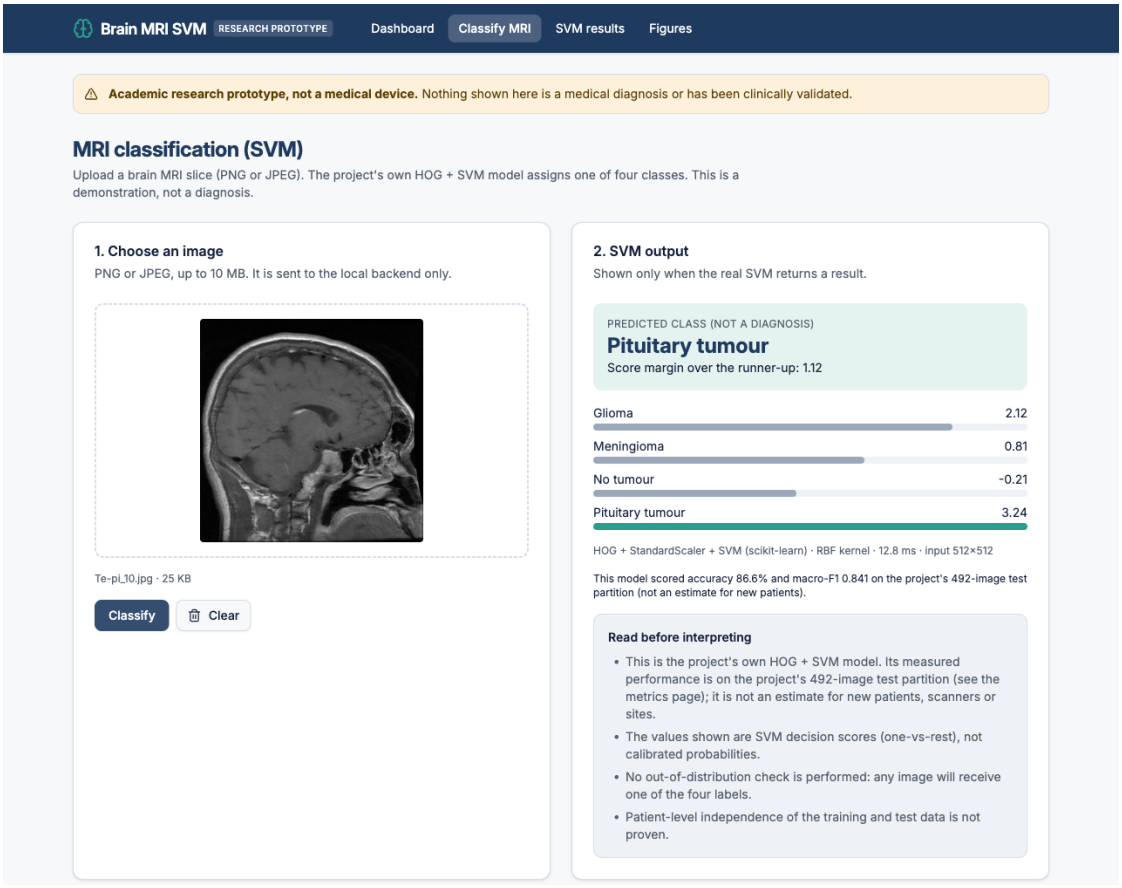


Figure 7.2: Classify MRI page with a dataset test image (the project's own HOG + SVM model, demonstration only).

The uploaded pituitary test image was classified as "Pituitary tumour" with a decision score of 3.24 against 2.12 for the runner-up (margin 1.12) in 12.8 ms. This is a plumbing check of the application, not an evaluation result, and the page says so; the scores are not probabilities.

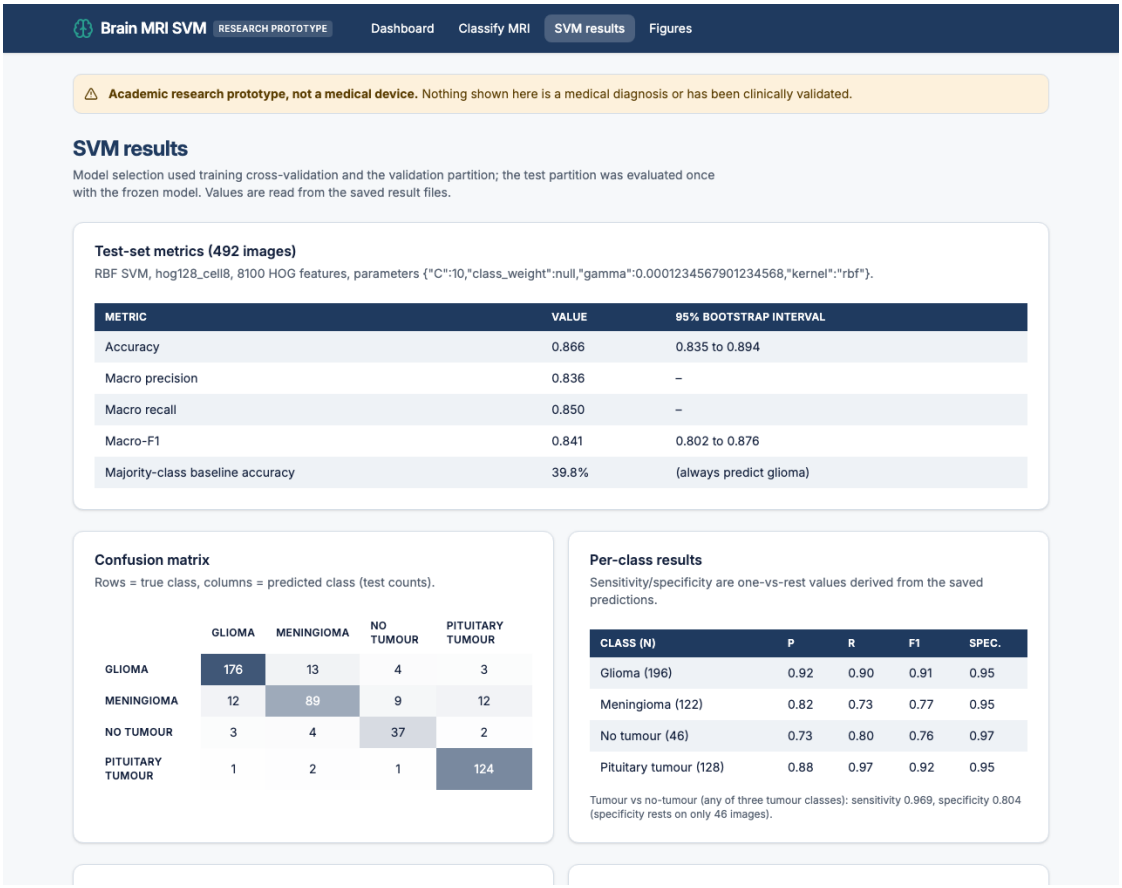


Figure 7.3: SVM results page: test-set metrics with bootstrap intervals, confusion matrix and per-class results read from the saved files.

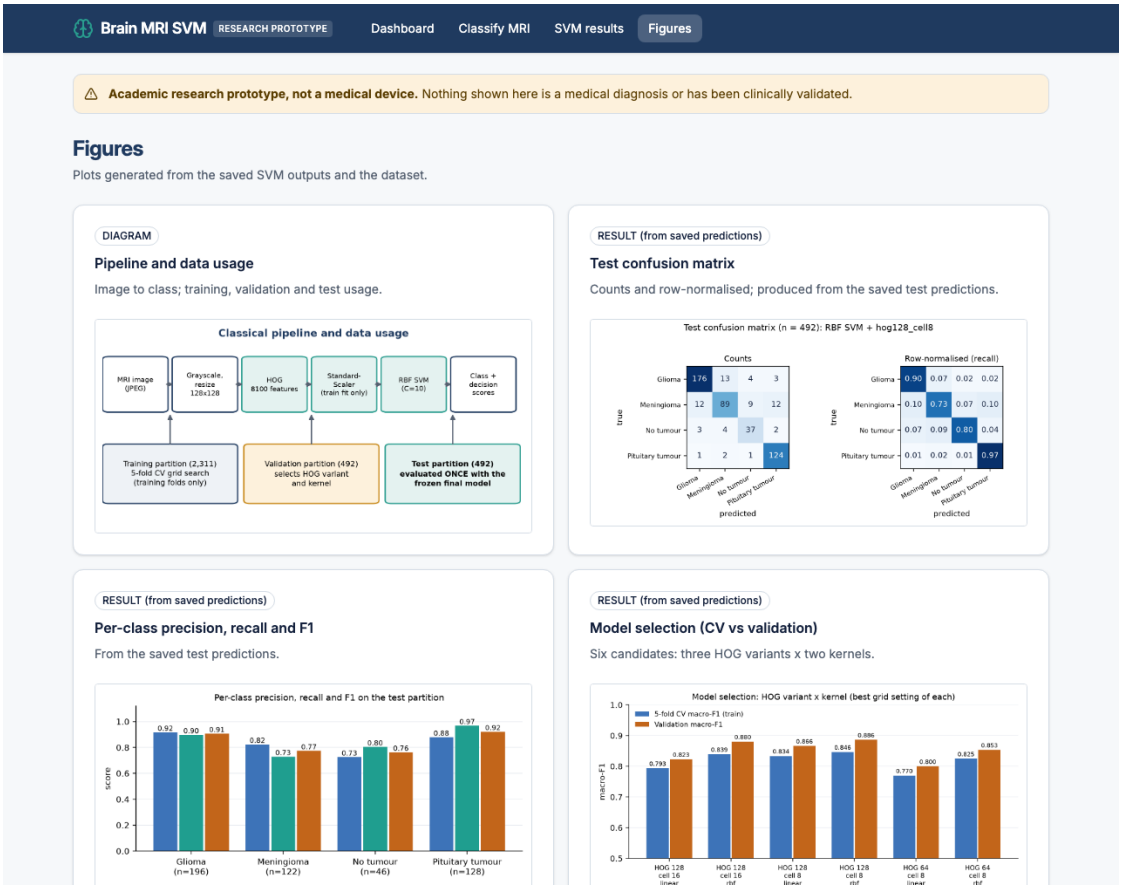


Figure 7.4: Figures page: saved figure sheets with diagram / result / data / audit badges.

7.2 Model Selection

Six candidate models were compared: three HOG variants (128 px with 16-px cells, 128 px with 8-px cells, 64 px with 8-px cells) × linear or RBF kernel. Hyper-parameters were tuned by 5-fold cross-validation on the training partition; the validation partition then chose the candidate. The test partition was not read.

Table 7.1: Model selection: best setting of each candidate (training CV, mean ± sd over 5 folds; validation partition).

HOG variant	Kernel	Features	Best parameters	CV macro-F1	Val. macro-F1	Val. accuracy
hog128_cell16	linear	1,764	C = 0.001, balanced	0.793 ± 0.010	0.823	0.829
hog128_cell16	rbf	1,764	C = 10, $\gamma = 1/n_f$	0.839 ± 0.006	0.880	0.886
hog128_cell8	linear	8,100	C = 0.001, balanced	0.834 ± 0.015	0.866	0.868
hog128_cell8	**rbf**	**8,100**	**C = 10, $\gamma = 1/n_f$**	**0.846 ± 0.015**	**0.886**	**0.900**
hog64_cell8	linear	1,764	C = 0.001, balanced	0.770 ± 0.008	0.800	0.817
hog64_cell8	rbf	1,764	C = 10, $\gamma = 1/n_f$	0.825 ± 0.015	0.853	0.868

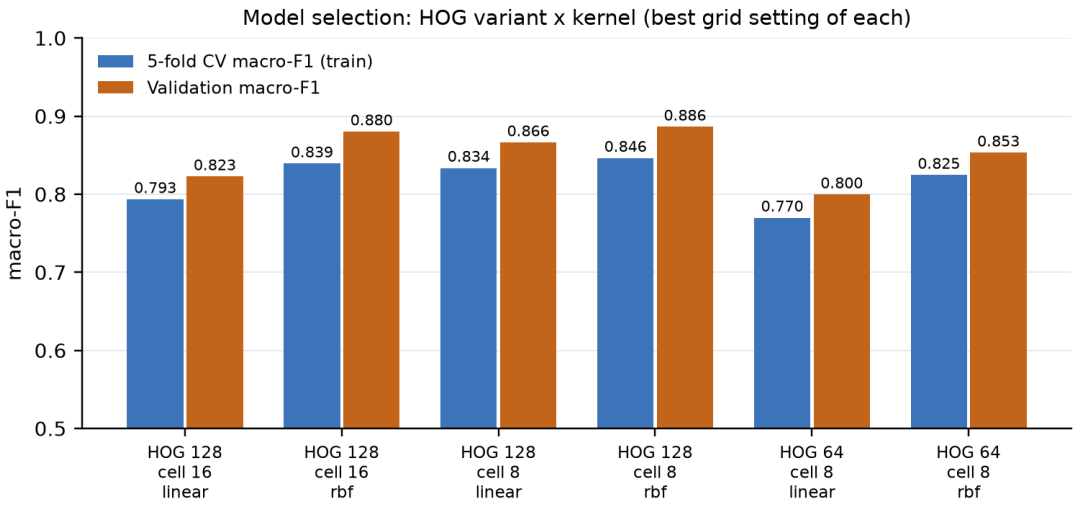


Figure 7.5: Cross-validation and validation macro-F1 of the six candidate models.

The RBF kernel beat the linear kernel for every HOG variant (validation macro-F1 0.880 vs 0.823, 0.886 vs 0.866 and 0.853 vs 0.800), finer 8-pixel cells beat 16-pixel cells, and 128-pixel images beat 64-pixel images. The RBF SVM on hog128_cell8 features had the highest validation macro-F1 (0.886) and was selected; the CV standard deviations (0.006–0.015) show that the training folds agree with this ranking.

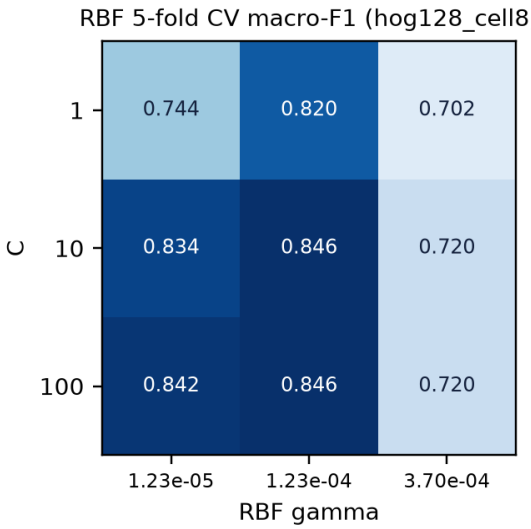


Figure 7.6: Cross-validated macro-F1 over the RBF hyper-parameter grid (training partition only).

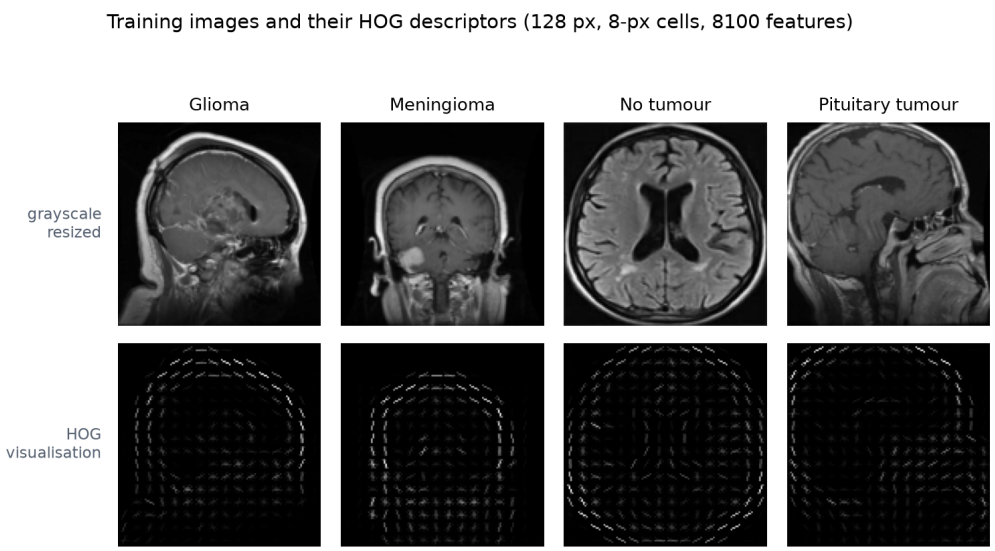


Figure 7.7: Training images and their HOG descriptors.

7.3 Test-Set Evaluation

The frozen model was evaluated once on the 492 test images (196 glioma, 122 meningioma, 46 no tumour, 128 pituitary).

Table 7.2: Test-set metrics of the selected RBF SVM (492 images; 95% bootstrap interval, 2,000 resamples).

Metric	Value	95% interval
Accuracy	0.866	0.835 to 0.894
Macro precision	0.836	–
Macro recall (= balanced accuracy)	0.850	–
Macro-F1	0.841	0.802 to 0.876
Weighted F1	0.864	–
Majority-class baseline accuracy (always glioma)	0.398	–

Table 7.3: Per-class results on the test partition (one-vs-rest specificity).

Class (n)	Precision	Recall	F1	Specificity
Glioma (196)	0.917	0.898	0.907	0.946
Meningioma (122)	0.824	0.730	0.774	0.949
No tumour (46)	0.725	0.804	0.763	0.969
Pituitary (128)	0.879	0.969	0.922	0.953

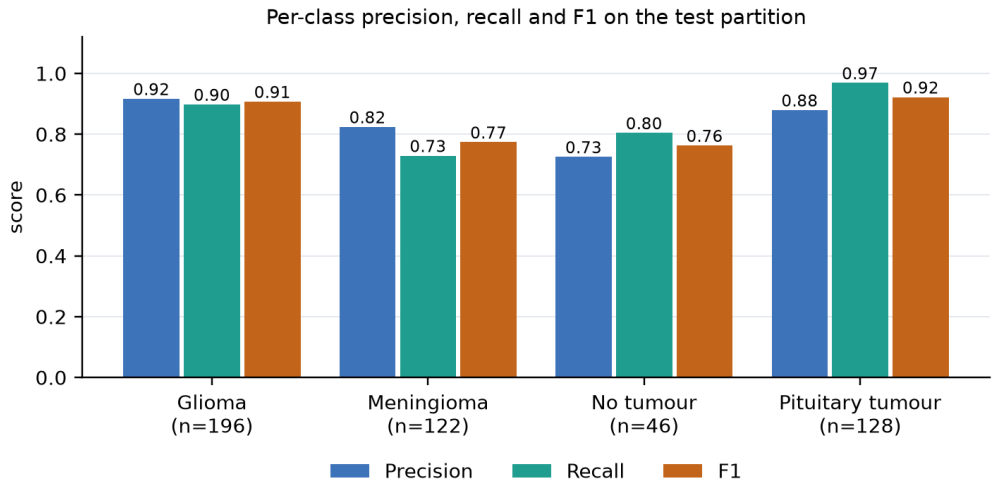


Figure 7.8: Per-class precision, recall and F1 on the test partition.

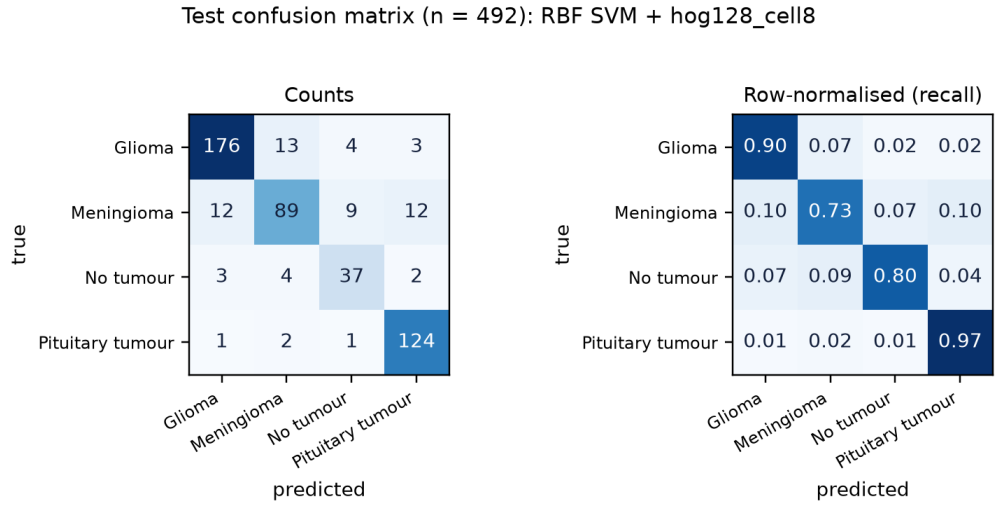


Figure 7.9: Test confusion matrix: counts (left) and row-normalised recall (right).

Table 7.4: Confusion matrix on the 492 test images (rows = true class, columns = predicted class).

True \ predicted	Glioma	Meningioma	No tumour	Pituitary
Glioma	176	13	4	3
Meningioma	12	89	9	12
No tumour	3	4	37	2
Pituitary	1	2	1	124

Table 7.5: Derived tumour vs no-tumour sensitivity and specificity.

Quantity	Value	Basis
Sensitivity (tumour found)	0.969	432 of 446 tumour images predicted as one of the three tumour classes
Specificity (no tumour kept)	0.804	37 of 46 no-tumour images predicted as no tumour

Pituitary tumours were recognised best (F1 0.922) and meningiomas worst (recall 0.730, with 12 predicted as glioma, 9 as no tumour and 12 as pituitary). The largest single confusion is glioma predicted as meningioma (13). Tumour-bearing images are detected with high sensitivity (0.969), while 9 of the 46 no-tumour images were given a tumour label (specificity 0.804). These values rest on only 46 no-tumour images and are not clinical estimates.

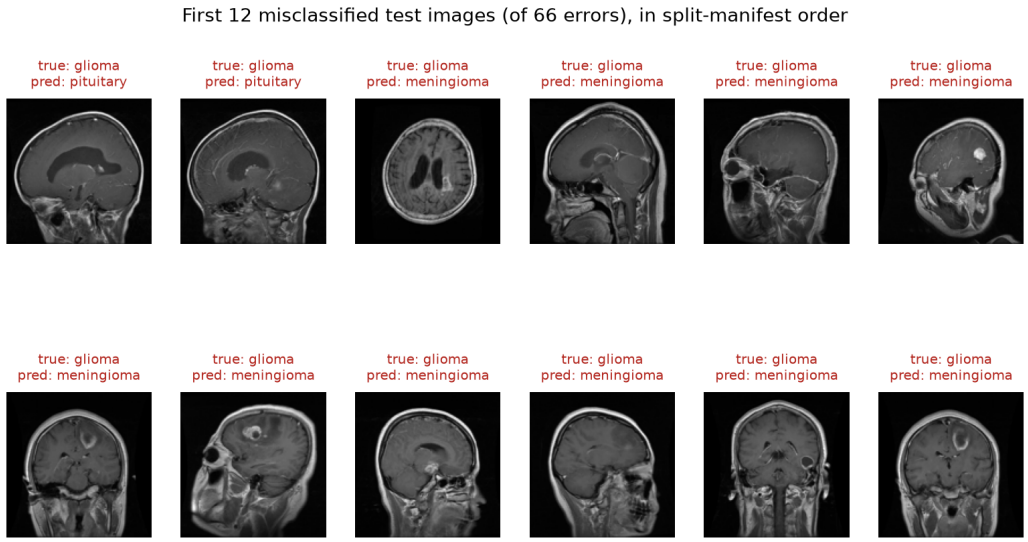


Figure 7.10: The first twelve misclassified test images (real dataset images, manifest order).

7.4 Leakage Check and What the Results Support

After selection and evaluation, the leakage controls were re-measured: 0 of 3,297 clusters span more than one partition and no byte-identical file is shared between partitions. The maximum 32×32 cosine similarity of any validation or test image to a training image is 0.980, below the 0.98 linking threshold, with a median of 0.944 (validation) and 0.937 (test); 188 test images still have a training image with similarity of 0.95 or more, so many held-out images have similar-looking training slices.

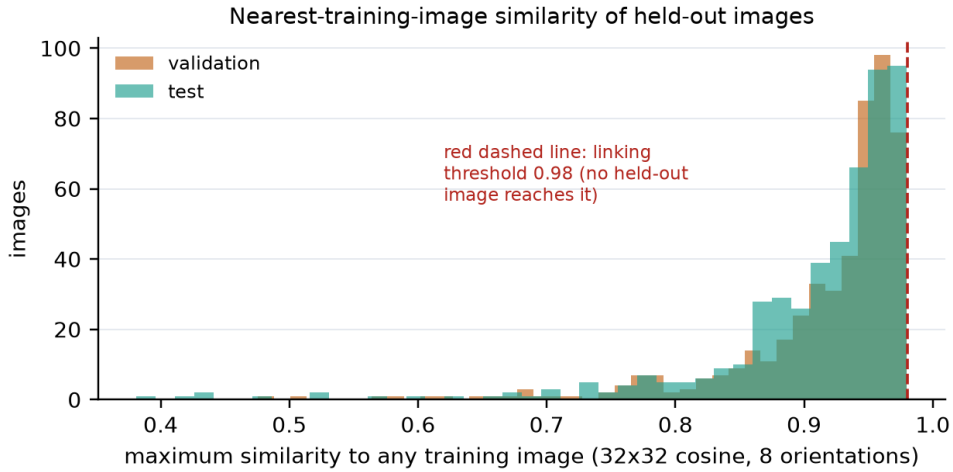


Figure 7.11: Nearest-training similarity of the validation and test images.

Table 7.6: Supported and unsupported statements.

Supported by the results	Not supported
The selected RBF SVM reaches 0.866 accuracy and 0.841 macro-F1 on the de-duplicated test set, far above the 0.398 baseline	That it will reach the same accuracy on new patients, scanners or sites
RBF beat linear, and finer HOG cells beat coarser ones, in cross-validation and on validation	That HOG + SVM is better than a deep-learning model (none was compared here)
The test set was used once, after the model was frozen	That the result is free of patient-level leakage
The split is protected against copy-level leakage (0 clusters cross)	That decision scores are calibrated probabilities
The application runs the real, hash-verified model and shows the saved results	Any use for diagnosis or treatment

8. Conclusions

8.1 Project Outcome

This project audited a public brain-tumour MRI dataset, found that its original test images were not independent of its training images, and rebuilt a cluster-disjoint split of 3,295 unique images. On that split a HOG + SVM pipeline was tuned with cross-validation and a validation partition, and the selected RBF SVM (8,100 HOG features, C = 10, γ = 1/8,100) was evaluated once on the untouched test partition: **accuracy 0.866** (95% interval 0.835–0.894) and **macro-F1 0.841** (0.802–0.876) against a majority-class baseline of 0.398, with a tumour-versus-no-tumour sensitivity of 0.969 and specificity of 0.804.

Following software-engineering practice, the work was specified (15 functional and 9 non-functional requirements), designed (architecture, five UML diagrams, algorithms, UI design), implemented and delivered as Brain MRI SVM, a tested React + TypeScript and FastAPI + scikit-learn application in which every requirement is traced to a test (13/13 backend tests, 7/7 smoke checks and 62/62 QA checks passed).

8.2 Key Takeaways

- Data auditing comes first: about two thirds of the original test images had a near-copy in training.
- A strong classical baseline needs no GPU: HOG + RBF SVM trains in seconds and classifies an image in about 12 ms.
- Selecting the model on validation data and reading the test set once keeps the reported score honest.
- Clear requirements, modular design, traceability and automated tests make a research prototype trustworthy.
- Honest reporting of limitations is part of responsible AI engineering.

8.3 Limitations

- Small, imbalanced test set (492 images, 46 no-tumour); one training run per configuration; bootstrap intervals describe test-sample variation only.
- Patient-level independence is unproven (no patient identifiers); near-duplicate detection is heuristic.
- HOG + SVM is a classical baseline on 2-D slices and was not compared with deep-learning models here.
- The SVM returns decision scores, not calibrated probabilities, and there is no out-of-distribution check: any image receives one of four labels.

8.4 Future Improvements and Scalability

- Use larger, more diverse datasets with patient identifiers so that splits can be made at patient level.
- Compare the SVM with convolutional and transfer-learning models on the same cluster-disjoint split, with more seeds and significance testing.

- Calibrate the scores into probabilities and add out-of-distribution detection and an abstain option.
- Add automated browser-level (end-to-end) tests and an accessibility audit.
- Scalability: containerise the backend with Docker and deploy on a cloud platform with authentication, logging and monitoring.
- Before any healthcare use: clinical validation, regulatory review, privacy assessment and clinical oversight.

9. References

[1] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273–297, 1995. doi:10.1007/BF00994018.

[2] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005. doi:10.1109/CVPR.2005.177.

[3] C.-W. Hsu, C.-C. Chang and C.-J. Lin, "A practical guide to support vector classification," technical guide, National Taiwan University (first version 2003; later revised). https://www.csie.ntu.edu.tw/~cjlin/

[4] C.-C. Chang and C.-J. Lin, "LIBSVM: a library for support vector machines," *ACM Transactions on Intelligent Systems and Technology*, vol. 2, no. 3, article 27, 2011. doi:10.1145/1961189.1961199.

[5] F. Pedregosa et al., "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] S. van der Walt et al., "scikit-image: image processing in Python," *PeerJ*, vol. 2, e453, 2014. doi:10.7717/peerj.453.

[7] E. I. Zacharaki et al., "Classification of brain tumor type and grade using MRI texture and shape in a machine learning scheme," *Magnetic Resonance in Medicine*, vol. 62, no. 6, pp. 1609–1618, 2009. doi:10.1002/mrm.22147.

[8] J. Cheng et al., "Enhanced performance of brain tumor classification via tumor region augmentation and partition," *PLoS ONE*, vol. 10, no. 10, e0140381, 2015. doi:10.1371/journal.pone.0140381.

[9] M. Nickparvar, "Brain Tumor MRI Dataset," Kaggle, Version 2, CC BY 4.0. https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset

[10] IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE, 1998.

[11] ISO/IEC/IEEE 29148:2018, *Systems and software engineering — Life cycle processes — Requirements engineering*, 2018.

[12] United Nations, "Transforming our world: the 2030 Agenda for Sustainable Development," 2015. https://sdgs.un.org/2030agenda